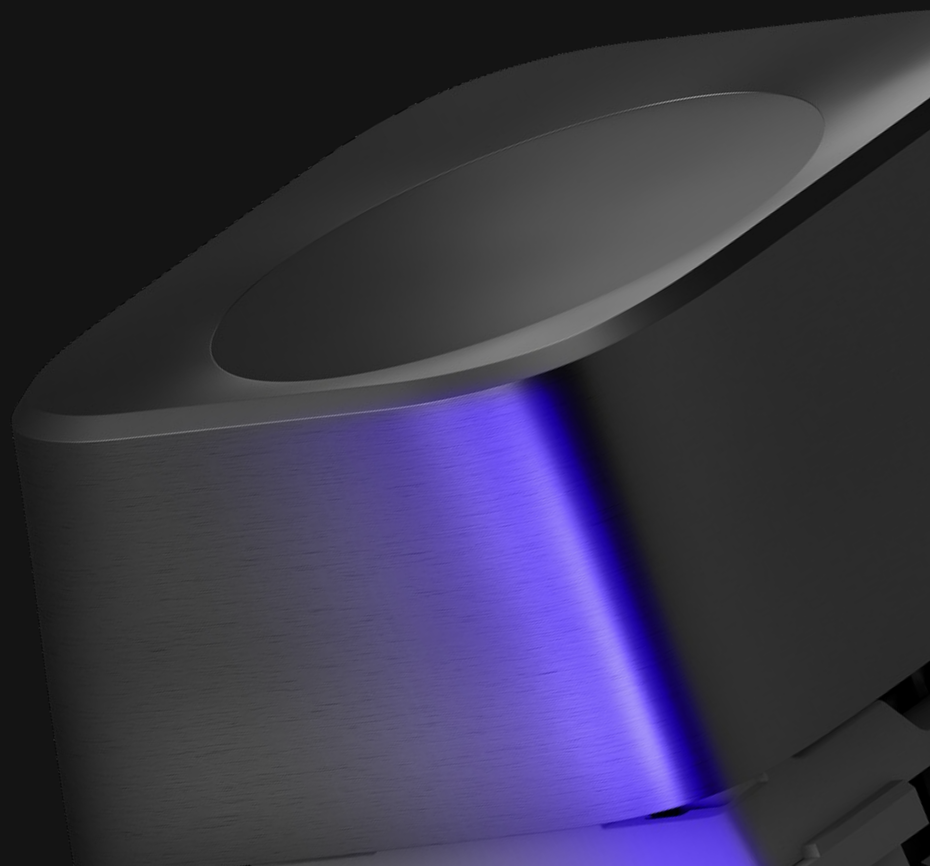
ЦЕНТРАЛЬНЫЙ УНИВЕРСИТЕТ

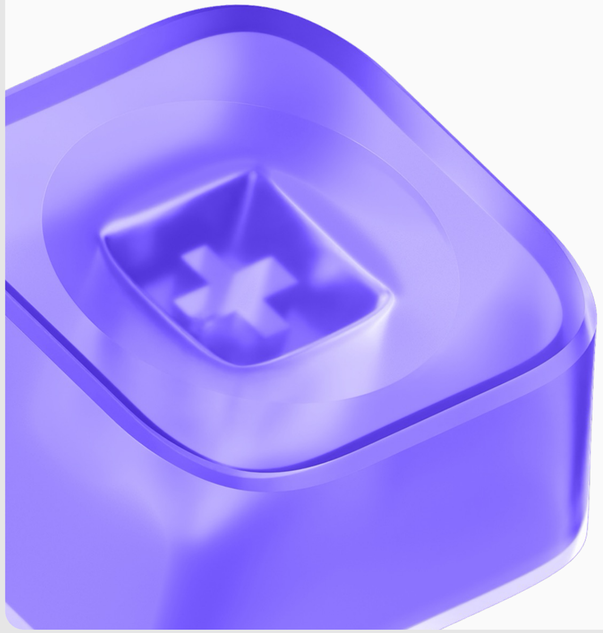
Случайный лес. Градиентный бустинг

ML
Машинное обучение

08



План лекции



СЛУЧАЙНЫЙ ЛЕС

01 Две случайности и предел усреднения

02 Пример, гиперпараметры, важность признаков

ГРАДИЕНТНЫЙ БУСТИНГ

01 Идея бустинга и пример по шагам

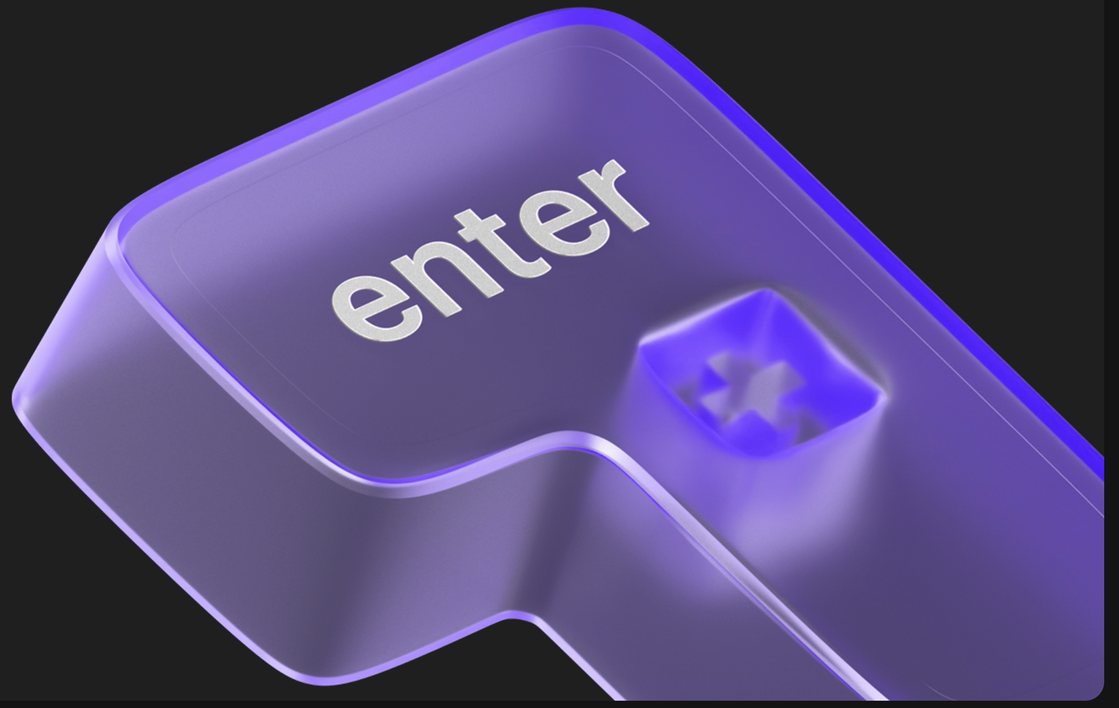
02 От остатков к антиградиенту

03 Значения листьев: парабола и шаг Ньютона

04 Классификация и сравнение с лесом

Часть I

Случайный лес



Ошибка модели: что уменьшает ансамбль

НАПОМИНАЕМ

$$\mathbb{E}\left[\left(y - a(x)\right)^2\right] = \sigma_\varepsilon^2 + \text{Bias}^2[a(x)] + \text{Var}[a(x)]$$

Шум σ_ε^2

Свойство данных. От выбора модели не зависит и не уменьшается ничем.

Смещение Bias^2

Насколько модель в среднем промахивается мимо истинной зависимости.

Разброс Var

Насколько ответ меняется при смене обучающей выборки.

Ансамбль уменьшает третье слагаемое и не трогает первые два. Отсюда требование к базовой модели: низкое смещение и высокий разброс.

Какую модель имеет смысл усреднять

Глубокое неограниченное дерево — модель с низким смещением и очень высоким разбросом.

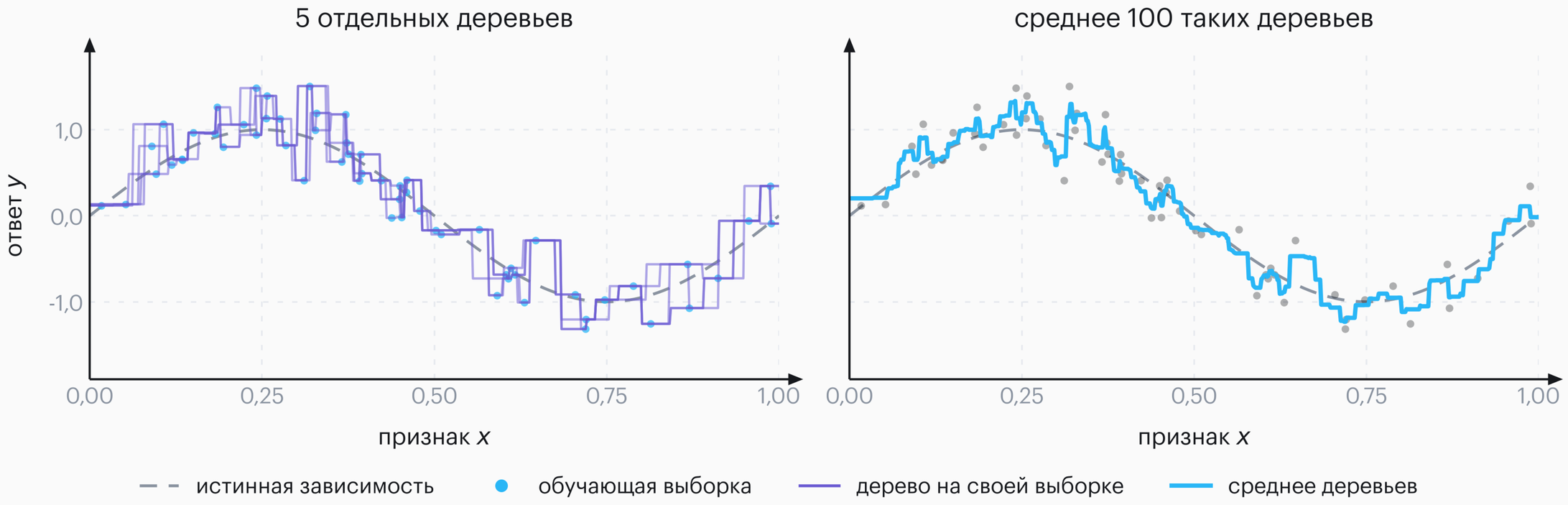
Глубокое дерево может иметь небольшое смещение, поскольку хорошо подстраивается под обучающую выборку.

Но стоит поменять одно наблюдение, и структура перестраивается целиком.

Такую модель и имеет смысл усреднять: разброс падает, смещение остаётся прежним.

Величина падения зависит не столько от числа моделей, сколько от того, насколько они не похожи друг на друга.

Разброс деревьев и их среднее



Отдельные деревья скачут от выборки к выборке, их среднее держится ровно.

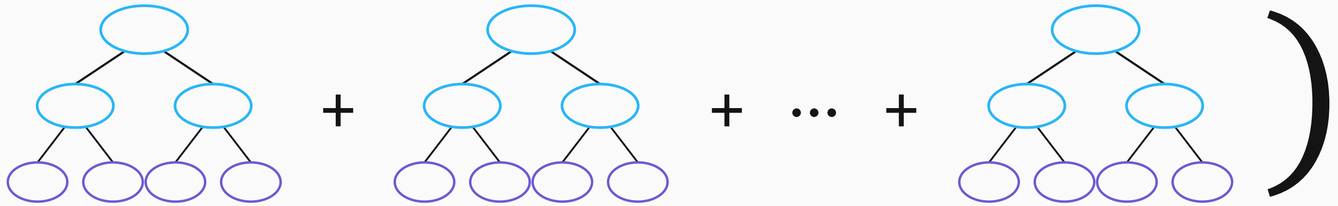
Случайный лес

Случайный лес (random forest, RF) — ансамбль деревьев, каждое из которых обучено на своей бутстрэп-выборке, а сплит в каждом узле ищется по случайным m признакам из p.

RF = бэггинг на деревьях + выбор m признаков в каждом сплите (узле)

отбор идёт заново перед каждым разбиением, а не один раз на дерево

$$\frac{1}{K}\left(\quad + \quad + \cdots + \quad \right)$$



m — сколько признаков перебирается в узле

p — сколько признаков всего

Алгоритм предложили Лео Брейман и Адель Катлер

Два источника случайности

Предсказываем стоимость квартиры по площади, метро, этажу, году постройки и ремонту

Случайность 1: какие объекты видит дерево

Бутстрэп-выборка того же размера, с возвращением

Исходная выборка

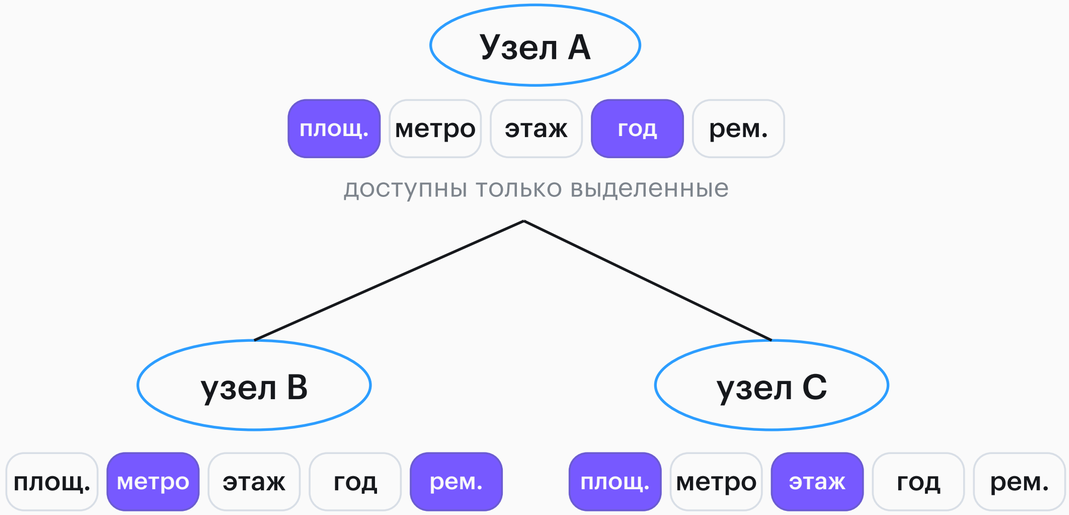
1	2	3	4	5

дерево 1	1	2	2	4	5	не попал: 3
дерево 2	1	3	3	4	5	не попал: 2
дерево 3	2	2	3	4	5	не попал: 1

Три дерева обучаются на трёх разных наборах объектов.

Случайность 2: какие признаки видит узел

Из p признаков в каждом узле случайно выбирается своё подмножество размера m

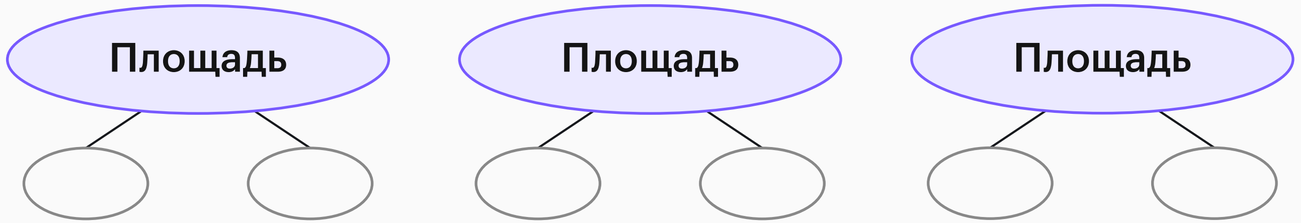


Набор кандидатов выбирается случайно заново перед каждым сплитом.

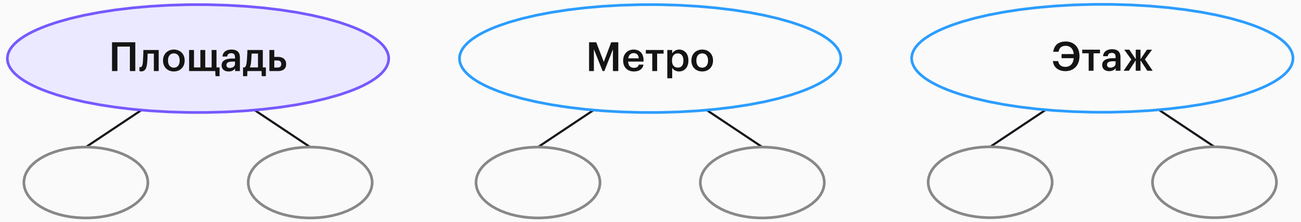
Зачем нужна вторая случайность

- Пусть один признак очень информативен, а остальные — умеренно.
- Тогда в бэггинге почти каждое дерево поставит его в корень.
- Верхушки совпадут, а вслед за ними и предсказания.
- При $p = 100$ и $m = 10$ сильный признак недоступен в девяти сплитах из десяти.

Бэггинг: сильный признак доступен в каждом узле



Случайный лес: в узле доступны m признаков из p



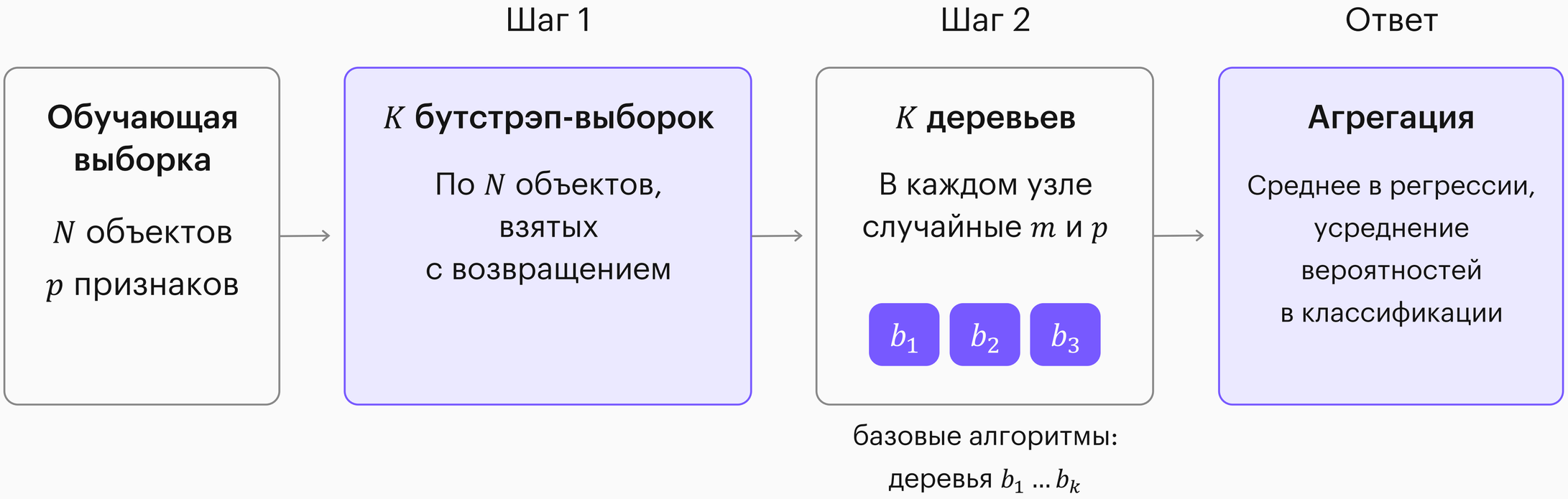
Декорреляция — снижение корреляции базисов ценой качества каждого из них по отдельности.

Одно дерево и лес на одной задаче

- Граница одного дерева сложена из прямоугольников и следует за отдельными объектами.
- У леса та же граница сглажена усреднением двухсот деревьев.
- Особенно показательно на шахматной доске, как области клеток становятся сглаженными.



Как обучается случайный лес



Каждый блок — строка алгоритма на следующем слайде.

Алгоритм случайного леса

Шаг 0

Задать число деревьев `n_estimators` = K.

Шаги $k = 1, \dots, K$

1. Взять бутстрэп-выборку размера `max_samples` из обучающей выборки.
2. Обучить дерево b_k на этой выборке.
   В каждом узле перебирать не все признаки, а случайные `max_features` из p.

Ответ:

Регрессия: $a(x) = \frac{1}{K}\sum_{k=1}^{K} b_k(x)$

Классификация: $a(x) = \arg\max_c \frac{1}{K}\sum_{k=1}^{K} p_{kc}(x)$

$p_{kc}(x)$ — вероятность класса c по дереву k.

Где здесь случайность

- Бутстрэп меняет выборку, на которой строится дерево.
- Подмножество признаков меняет сплит в каждом узле.

Что из этого следует

Два источника случайности дают разные деревья, а разные деревья дают низкую корреляцию ответов. Именно корреляция определяет **предел усреднения**.

Предел усреднения

$$\mathrm{Var}[a(x)] = \rho\sigma^2 + \frac{1-\rho}{K}\sigma^2$$

σ^2 — дисперсия ответа одного дерева, ρ — корреляция ответов двух деревьев, K — число деревьев

в предположении одинаковой дисперсии деревьев и одинаковой попарной корреляции их ответов

$$\lim_{K\to\infty} \rho\,\sigma^2 = \rho\sigma^2$$

Первое слагаемое от числа деревьев не зависит.

$$\lim_{K\to\infty} \frac{1-\rho}{K}\sigma^2 = 0$$

Второе слагаемое убывает к нулю с ростом числа деревьев.

Разброс ансамбля при разных p

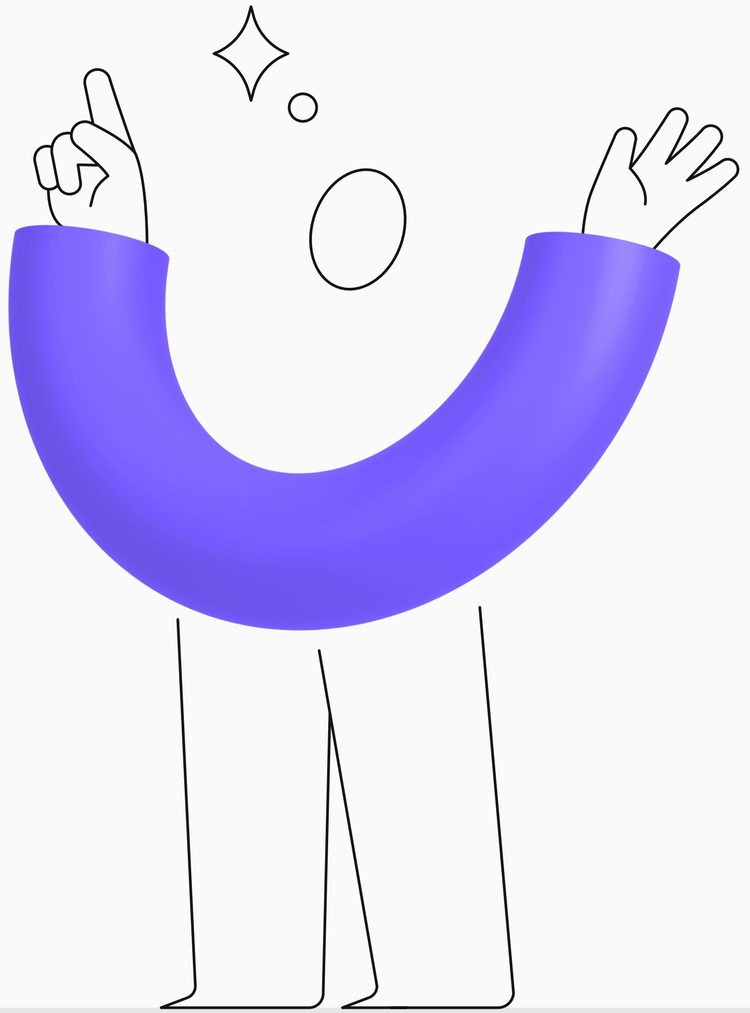


$$\mathrm{Var}[a(x)] \underset{K\to\infty}{\to} \rho\sigma^2$$

Разброс не опустится ниже $\rho\sigma^2$, нижнюю границу задаёт корреляция деревьев.

Лес на данных

Пример, гиперпараметры, важность признаков



Считаем лес вручную

Пять квартир, два признака — площадь и время до метро, предсказываем цену.

Лес из трёх пней, в единственном узле доступен один случайный признак.

Данные

№	Площадь, м²	До метро, мин	Цена, млн ₽
1	30	5	6
2	40	8	8
3	50	25	7
4	60	3	12
5	70	10	12

Дерево 1: бутстрэп индексы {1, 2, 2, 4, 5}, случайный признак — площадь

Порог	Левый лист	Правый лист	SSE
$x_1 \le 35$	6	8, 8, 12, 12	16
$x_1 \le 50$	6, 8, 8	12, 12	2,67
$x_1 \le 65$	6, 8, 8, 12	12	19

SSE (sum of squared errors) — сумма квадратов отклонений от среднего своего листа.

Дерево	Признак	Порог	SSE
1	площадь	$x_1 \le 35$	2,67
2	до метро	$x_2 \le 4$	22
3	площадь	$x_1 \le 55$	0,67

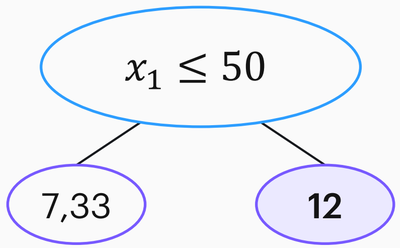
Лес: второе дерево хуже, ему достался признак, который объясняет цену слабо.

Три дерева и ответ леса

Новая квартира: 55 м², 12 минут до метро. Настоящая цена — 10 млн ₽

Дерево 1

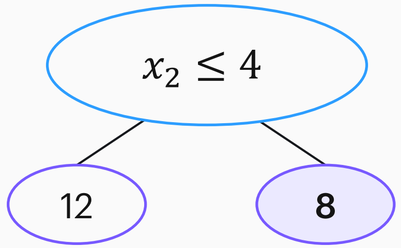
бутстрэп {1, 2, 2, 4, 5}, случайный признак «площадь»



SSE на обучении	2,67
Предсказание для новой квартиры	12
Ошибка на новой квартире	+2,00

Дерево 2

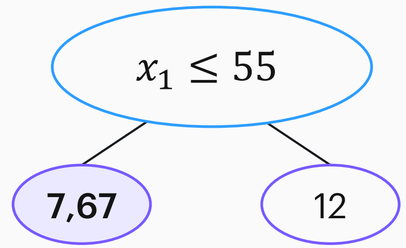
бутстрэп {1, 3, 3, 4, 5}, случайный признак «метро»



SSE на обучении	22,00
Предсказание для новой квартиры	8
Ошибка на новой квартире	−2,00

Дерево 3

бутстрэп {2, 2, 3, 4, 5}, случайный признак «площадь»



SSE на обучении	0,67
Предсказание для новой квартиры	7,67
Ошибка на новой квартире	−2,33

Ответ леса: $(12 + 8 + 7{,}67)/3 = 9{,}22$

Ошибка леса: −0,78

Ансамбль оказался точнее каждого из трёх деревьев, потому что их ошибки направлены в разные стороны и при усреднении гасят друг друга.

OOB-оценка

OOB-оценка (out-of-bag estimate) — оценка качества по объектам, не попавшим в бутстрэп-выборку.

Каждый объект предсказывают только деревья, которые его не видели.

$$\mathrm{OOB} = \frac{1}{N}\sum_{i=1}^{N} L\left(y_i, \underbrace{\frac{1}{|\{k: x_i \in \mathrm{OOB}_k\}|}\sum_{k:x_i \in \mathrm{OOB}_k} b_k(x_i)}_{\text{средний ответ деревьев, не видевших объект}}\right)$$

OOB_k — объекты, не попавшие в бутстрэп-выборку дерева b_k.

Усреднение идёт по деревьям, для которых $x_i \in \mathrm{OOB}_k$.

В sklearn при `oob_score=True` атрибут `oob_score_` содержит OOB-качество: по умолчанию долю верных ответов в классификации и R^2 в регрессии.

Можно использовать OOB вместо отдельной валидационной выборки и подбирать по ней гиперпараметры. После выбора модели итоговое качество оценивают на независимой тестовой выборке.



Отдельного прогона оценка не требует: она считается попутно с обучением, тогда как кросс-валидация по пяти блокам обучает лес пять раз.

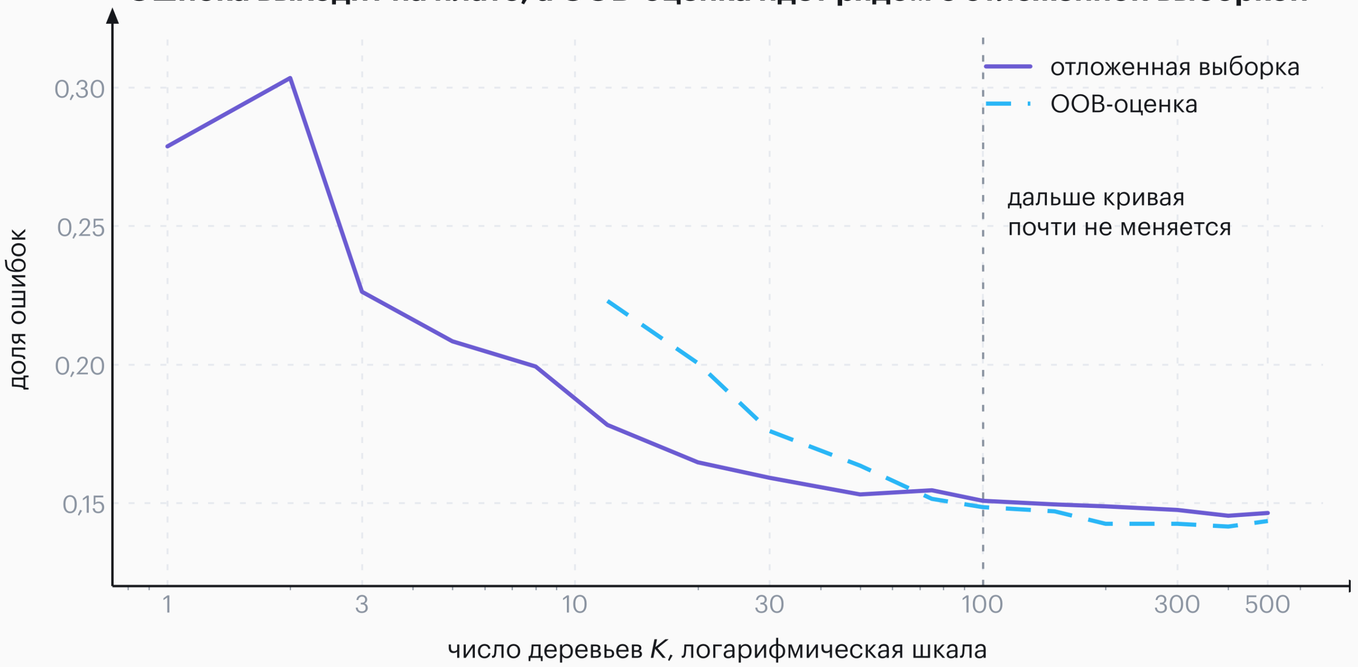
Гиперпараметры: n_estimators

Как настраивать

- Рост числа деревьев не ухудшает качество систематически: кривая выходит на плато.
- Брать K с запасом и добавлять деревья, пока ошибка не вышла на плато.
- За выходом на плато можно следить по OOB-оценке, без отложенной выборки.

Платой за большое K становятся время обучения и размер модели.

Ошибка выходит на плато, а OOB-оценка идёт рядом с отложенной выборкой



Ошибка падает с 0,28 при одном дереве до 0,15 к пятидесяти и дальше почти не меняется. OOB-оценка идёт рядом с отложенной выборкой.

OOB-оценка показана с $K = 10$: при меньшем числе деревьев часть объектов не попадает в OOB ни разу, и sklearn засчитывает им первый класс из classes_.

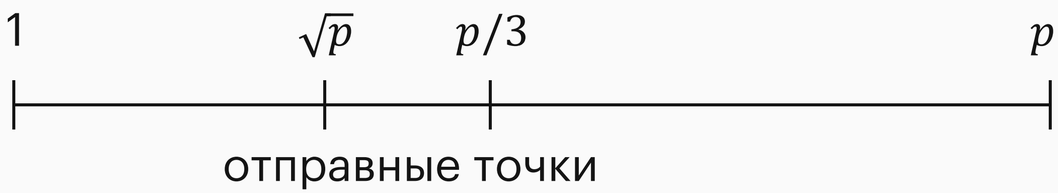
Гиперпараметры: max_features

Как настраивать

- Самый важный гиперпараметр леса: он задаёт корреляцию деревьев, а с ней и нижнюю границу ошибки.
- Отправные точки — $\sqrt{p}$ в классификации и $p/3$ в регрессии.
- У RandomForestRegressor по умолчанию берутся все p, то есть без явного указания получается бэггинг.

Настройку леса начинают с этого параметра, а часто им же и заканчивают.

Диапазон значений



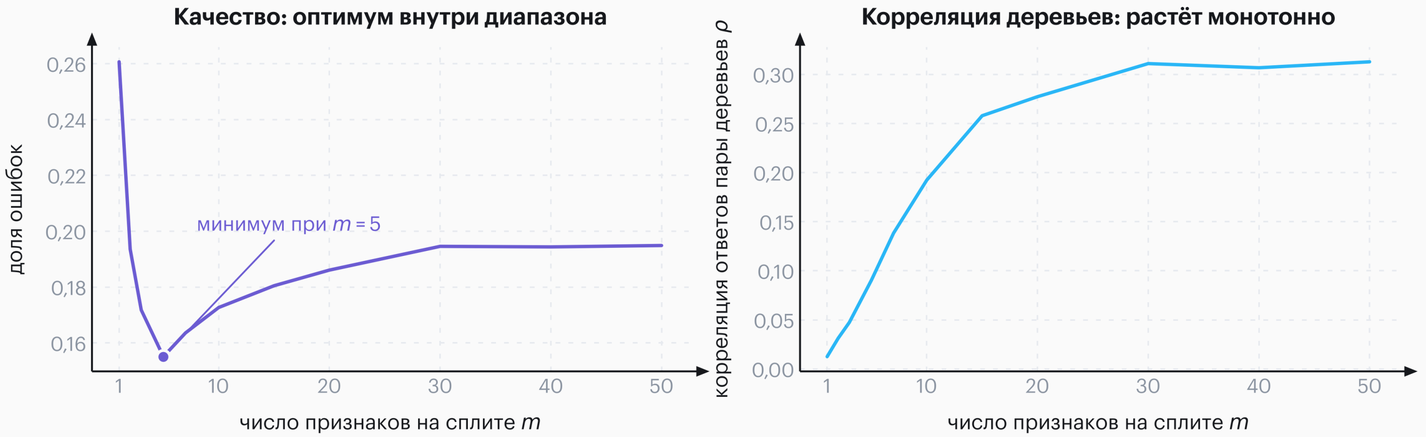
При $p = 50$ отправные точки — семь признаков в классификации и семнадцать в регрессии.

На левом краю деревья меньше всего похожи друг на друга, но каждое слабее. На правом отбор не работает, и лес превращается в бэггинг.

Почему max_features не уменьшают до единицы

Уменьшение m

- ↓ снижает корреляцию деревьев, и это желаемый эффект
- ↑ увеличивает разброс отдельного дерева: структура сильнее зависит от случайного отбора
- ↑ увеличивает его смещение: информативный признак чаще оказывается недоступен в том узле, где он был бы лучшим



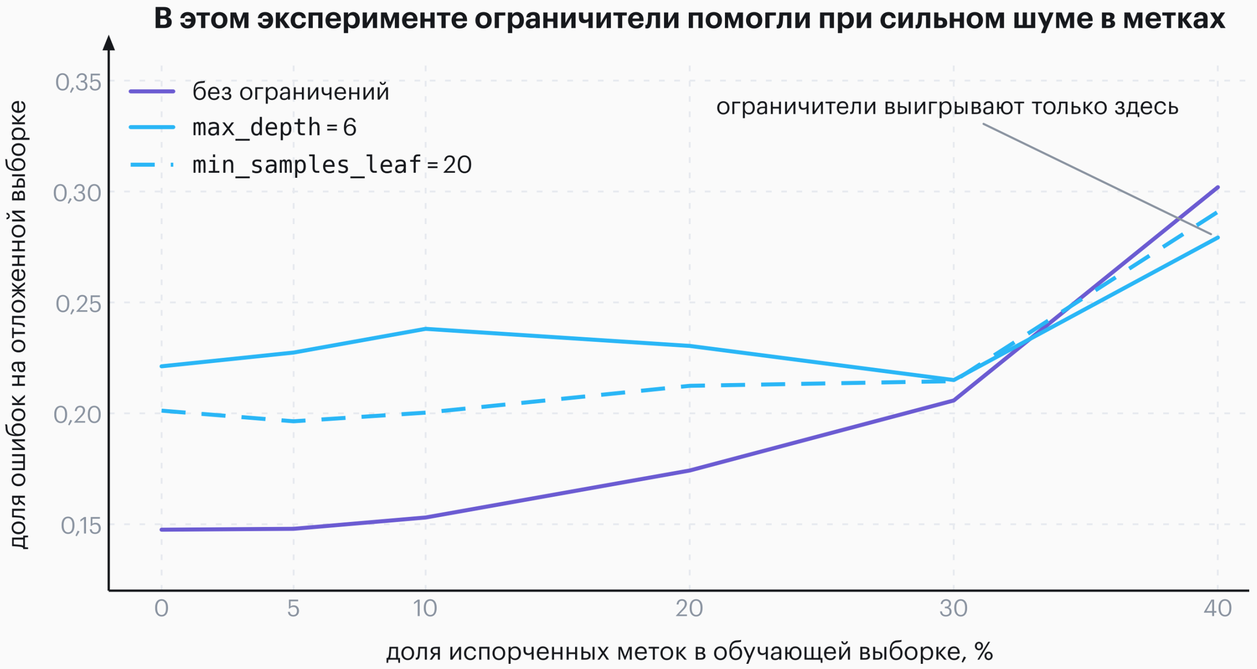
$p = 50$, информативных 10

Корреляция растёт с m монотонно, от 0,013 до 0,313, а ошибка минимальна при $m = 5$.

Гиперпараметры: max_depth и min_samples_leaf

Как настраивать

- Ограничивать глубину имеет смысл при сильном шуме в метках и ради времени обучения на больших выборках.
- `min_samples_leaf` действует более локально, чем `max_depth`, — ограничивает размер отдельных листьев, а не глубину всего дерева.



На чистых данных 0,147 без ограничений против 0,221 при max_depth = 6. Выигрыш появляется при 40% шума: 0,279 против 0,302.

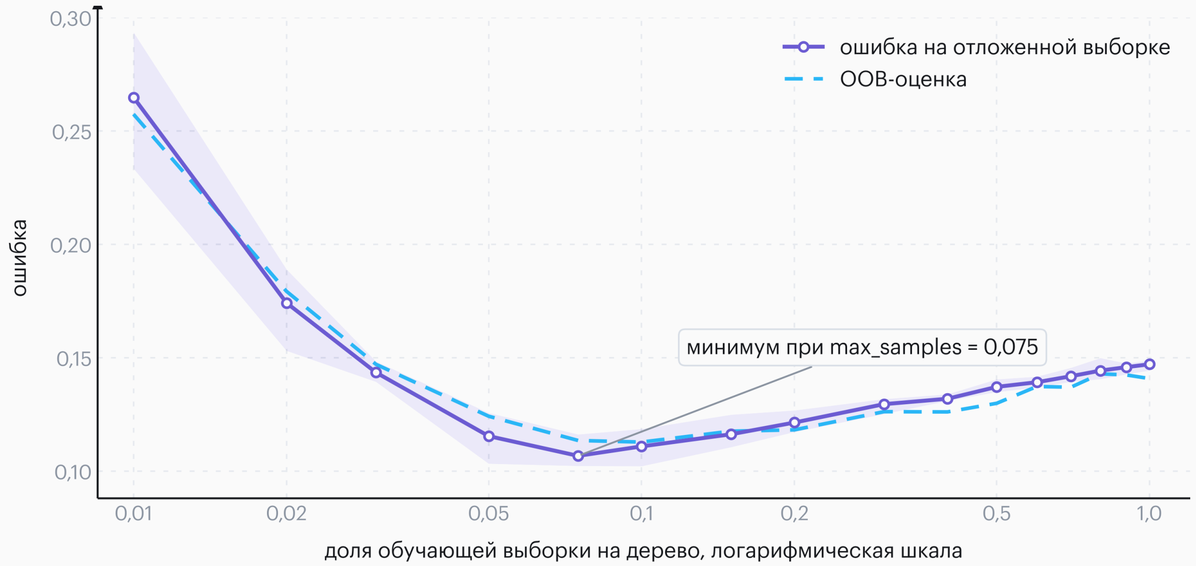
Деревья в лесу можно изначально не ограничивать, а глубину и размер листа подбирать по валидации.

Гиперпараметры: max_samples

Как настраивать

- При `bootstrap=True` размер выборки на дерево задаёт `max_samples`.
- Меньшая доля снижает корреляцию деревьев и ускоряет обучение, но каждое дерево видит меньше объектов.
- При `bootstrap=False` каждое дерево видит всю выборку, и OOB-оценка становится недоступна.

`max_samples` — доля обучающей выборки, которая идёт на одно дерево при **`bootstrap=True`**.



Шкала логарифмическая, коридор — разброс пяти прогонов. Минимум ошибки 0,107 при доле 0,075 против 0,147 на полной выборке.

Гиперпараметры: итог

Параметр	Что меняет	С чего начинать
`max_features`	корреляцию деревьев	отправные точки $\sqrt{p}$ и $p/3$
`n_estimators`	устойчивость среднего	столько, сколько позволяет время
`max_depth` и `min_samples_leaf`	смещение отдельного дерева	без ограничений
`max_samples`	корреляцию и скорость обучения	вся выборка

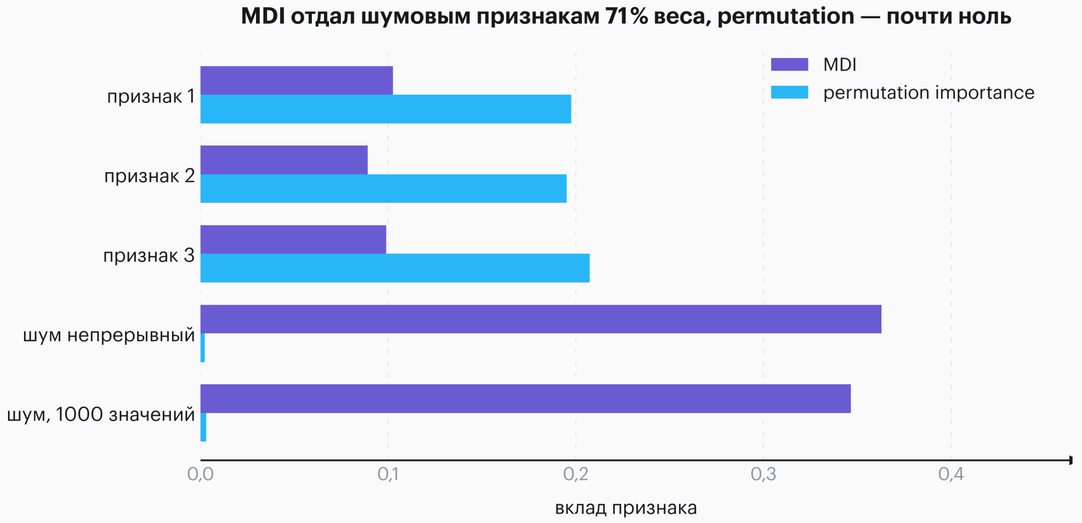
Настройку леса начинают с **`max_features`**, число деревьев увеличивают до стабилизации качества. Глубину, размер листа и долю объектов подбирают по валидации, с оглядкой на время обучения и размер модели.

Важность признаков

$$\mathrm{MDI}(f) = \frac{1}{K}\sum_{k=1}^{K}\sum_{v \in V_k(f)}\frac{N_v}{N}\Delta H_v$$

$V_k(f)$ — узлы дерева k, где сплит сделан по признаку f, N_v — число объектов в узле v, ΔH_v — падение неоднородности при сплите.

- Формула введена на неделе 6: в лесу добавилось внешнее усреднение по K деревьям.
- **Permutation importance** — рост ошибки на отложенной выборке после перемешивания значений признака.
- MDI считается на обучающей выборке и растёт с числом порогов, поэтому шуму и достаётся вес. Permutation importance такого не даёт, зато занижает важность взаимозаменяемых.



Метка зависит от трёх бинарных признаков. Двум добавленным шумовым MDI отдал 71% веса, permutation importance — 0,002 и 0,003.

Случайный лес: итоги

Преимущества

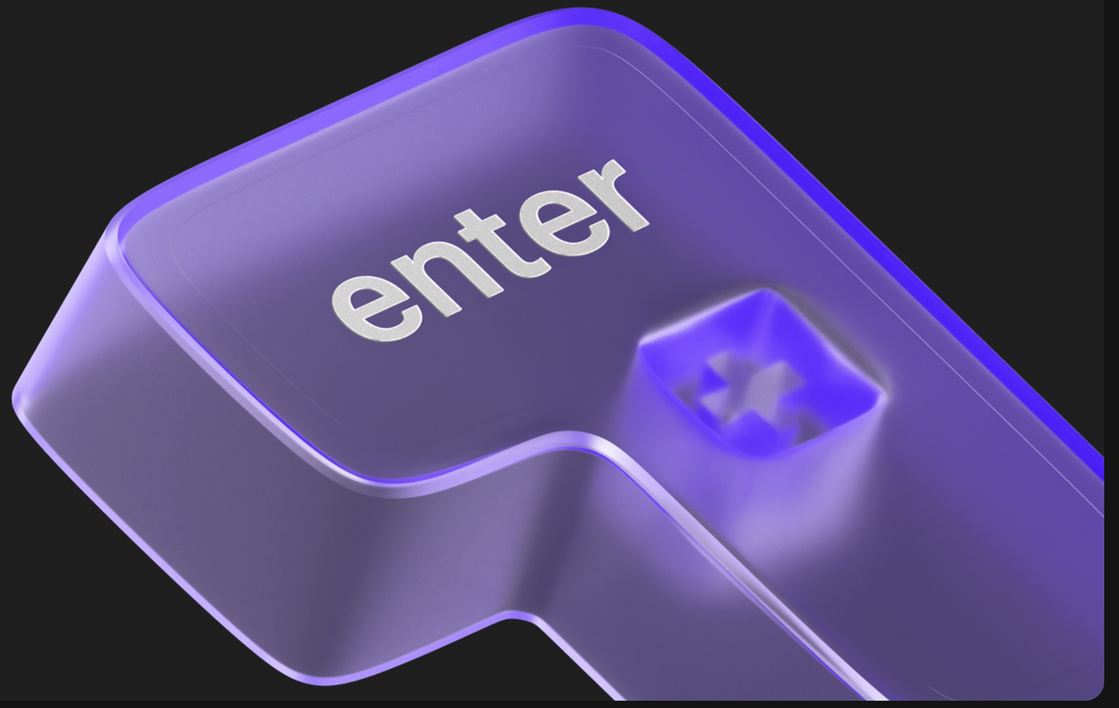
- Часто даёт сильный baseline при небольшой настройке.
- Оценка обобщающей ошибки получается попутно, по OOB.
- Шкалы признаков не важны, нормализация не нужна.
- Обучение раскладывается по ядрам почти линейно.

Ограничения

- Интерпретируемости: путь к конкретному ответу не проследить.
- Стоимости инференса: память и время растут пропорционально K.
- Экстраполяции: за диапазон обучающих ответов лес не выходит.
- Смещения: усреднение его не уменьшает при любом K.

Часть II

Градиентный бустинг



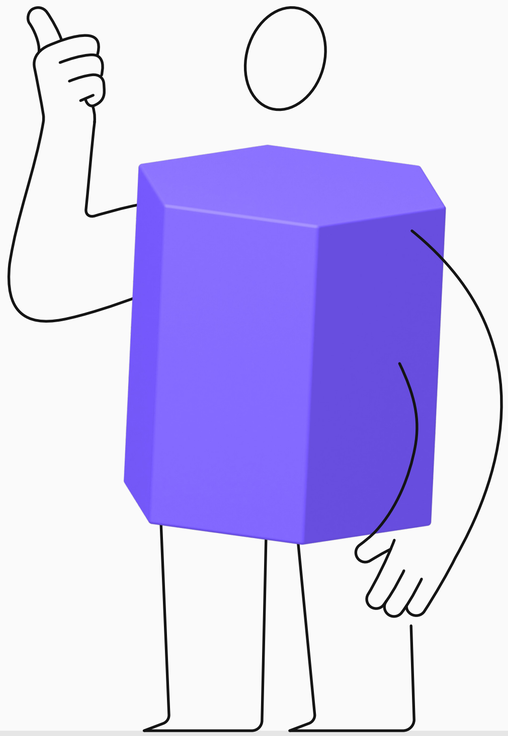
Ограничение, которое не снять усреднением

$$\mathrm{Bias}[a(x)] = \mathrm{Bias}[b(x)]$$

Математическое ожидание среднего равно среднему математических ожиданий, а деревья смещены одинаково.

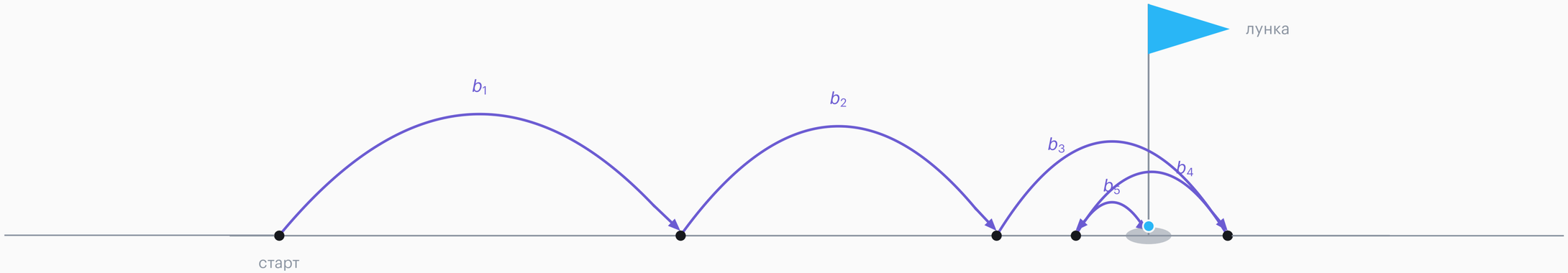
Пока базовое дерево почти несмещённое, лес работает отлично. Но там, где сильную и несмещённую модель построить не получается, добавление деревьев не даёт прироста качества.

Что, если каждая следующая модель будет учиться на ошибках предыдущих?



Бустинг

Бустинг (boosting) — метод построения ансамбля, при котором несколько базовых моделей последовательно объединяются в одну более сильную. Каждая следующая модель строится с учётом ошибок предыдущих.

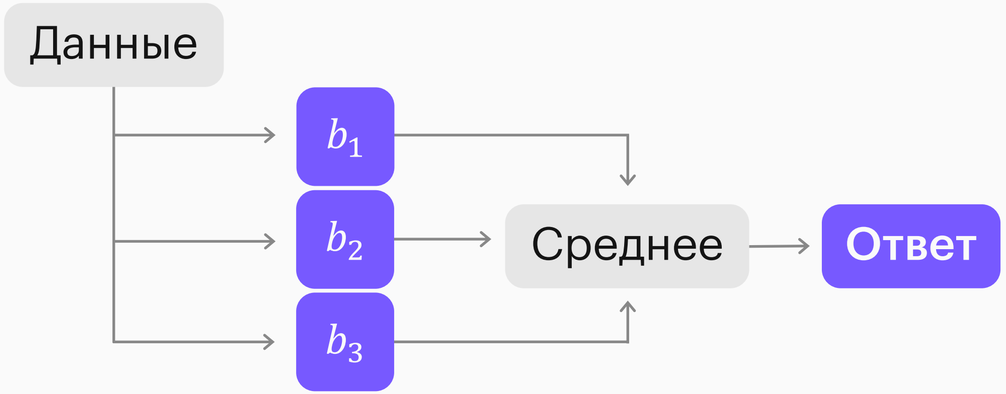


Игрок в гольф не пытается попасть одним ударом. Итоговое положение мяча определяется не лучшим ударом и не средним из них, а их суммой.

Наиболее распространённый вариант сегодня — градиентный бустинг (gradient boosting, GB), его предложил Джером Фридман в 1999 году.

Усреднение против сложения

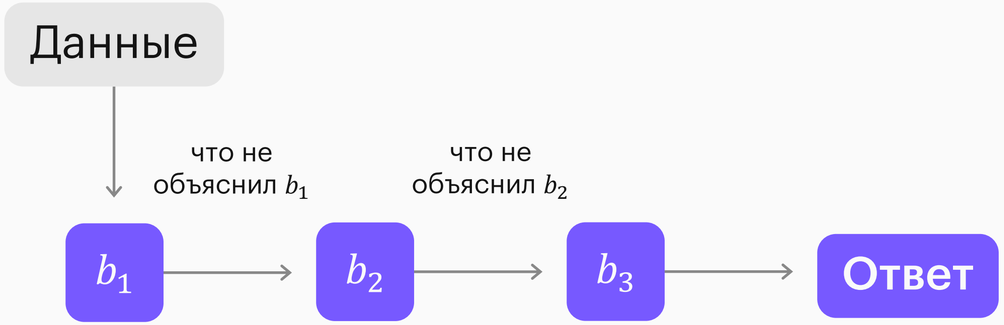
Случайный лес



$$a(x) = \frac{1}{K}\sum_{k} b_k\,(x)$$

Модели не знают друг о друге, порядок не важен

Градиентный бустинг



$$a_k(x) = a_{k-1}(x) + \eta b_k(x)$$

Каждая модель видит работу предыдущих, порядок существен

В лесу базовыми делают глубокие деревья с небольшим смещением. В бустинге требования к смещению отдельной модели ниже: предсказание уточняют последующие модели.

Формула ансамбля и темп обучения

$$a_k(x) = a_{k-1}(x) + \eta b_k(x)$$

a_{k-1} — ансамбль, накопленный за первые $k-1$ шагов, b_k — очередной базис, η — темп обучения

Базис подобран под конечную выборку и вместе с сигналом поймал долю шума. Прибавляя его целиком, ансамбль забирает и то, и другое.

Поправка домножается на небольшое η и входит укороченным шагом.

Темп обучения η **(learning rate, shrinkage)** берётся в интервале от нуля до единицы, на практике около 0,1.

Платой становится число шагов: чем меньше η, тем больше базисов нужно. Их настраивают вместе.

Почему дерево подходит как базис

$$b_k = \arg\min_b \sum_{i=1}^{N} L\left(y_i, a_{k-1}(x_i) + b(x_i)\right)$$

Пространство поиска — множество всех функций, и полный перебор в нём невозможен.

Поэтому оптимизацию проводят внутри выбранного класса базовых моделей.

Дерево удобно сразу по трём причинам

- Кусочно-постоянно и потому задаёт поправку на всём пространстве признаков, а не только в обучающих точках.
- Дерево не требует масштабирования входных признаков.
- Подстроится почти под любой набор поправок, если сгруппировать похожие объекты в один лист.

Что разберём дальше

01

Пример на регрессии

Три пня на пяти квартирах. Частный случай — регрессия и квадратичная ошибка, каждая величина считается в одну строку.

02

Общий алгоритм

Остатки заменяются антиградиентом, и метод перестаёт зависеть от того, какая функция потерь выбрана.

03

Значения листьев

Разложение потерь до второго порядка позволяет приблизить оптимальное значение листа формулой — G/H.

04

Классификация

Те же формулы в шкале логитов. Меняется не алгоритм, а то, что в него подставляется.

От частного примера к общей математической постановке.

Данные сквозного примера

Пять квартир: площадь и цена

№	Площадь, м²	Цена, млн ₽
1	30	6
2	40	8
3	50	7
4	60	12
5	70	12

Задача: регрессия

Предсказывается цена, то есть вещественное число.

Мера качества: квадратичная ошибка

Градиентный бустинг на регрессии считается проще всего, так как при квадратичной ошибке каждое следующее дерево обучается на остатках (невязках).



Пень делает одно разбиение по площади. Перебираются пороги 35, 45, 55 и 65 — середины между соседними площадями.

Пример: шаг 0

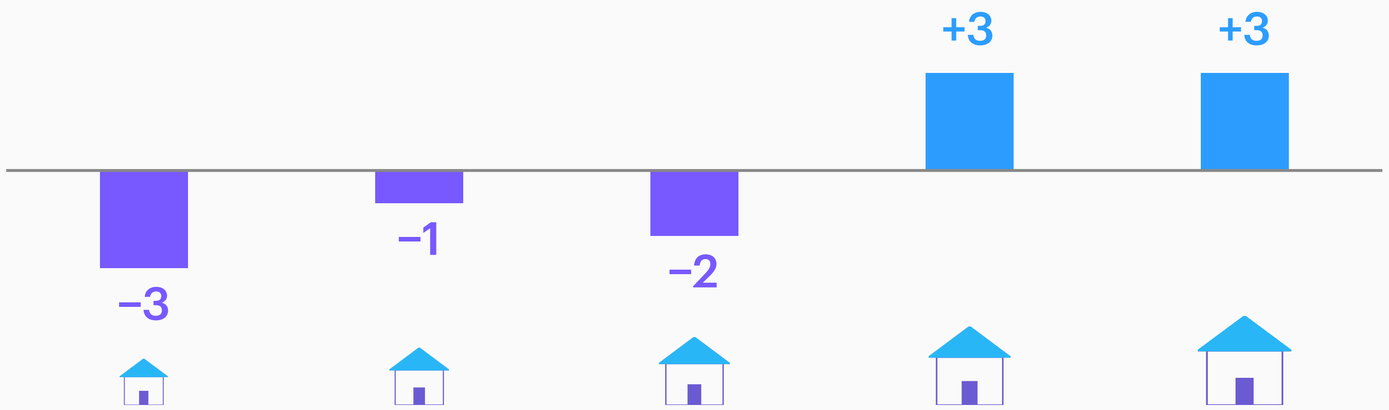
$a_0 = \bar{y} = 9$

лучшая константа

ошибка 6,4

первое приближение:
одна цена для всех
пяти квартир

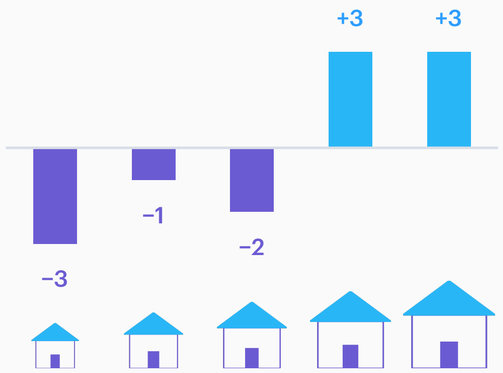
остатки $r_i = y_i - a_0(x_i)$



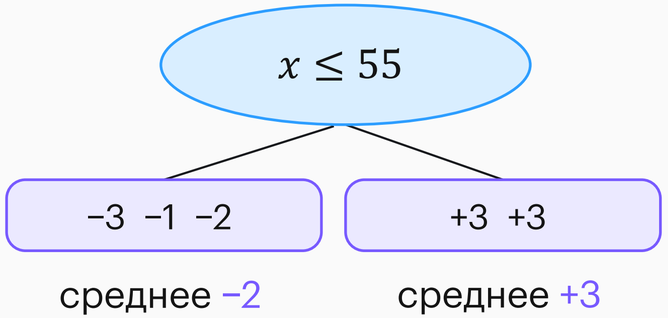
Итерация 1: дерево b_1 учится на остатках

На чём учится дерево

$$r_i = y_i - a_0(x_i)$$



Дерево b_1



В лист кладётся среднее остатков

Ансамбль после шага

$$a_1 = a_0 + \eta b_1$$

Ошибка была **6,4**

Стала **1,9**

Квартира 30 м²

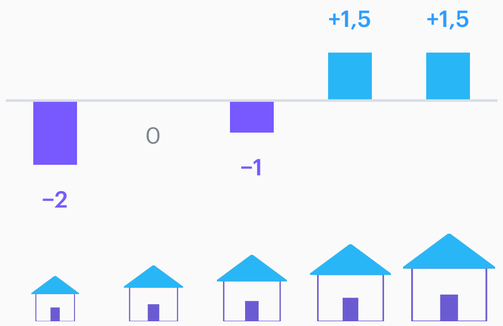
$$9 - 0{,}5 \cdot 2 = 8$$

Целевой переменной служит столбец остатков, а столбца цен дерево не видит вообще.

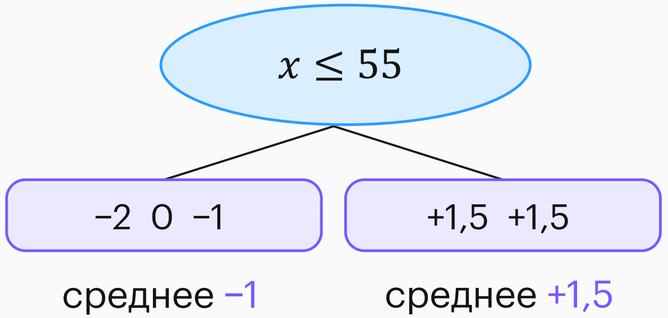
Итерация 2: дерево b_2 на новых остатках

На чём учится дерево

$$r_i = y_i - a_1(x_i)$$



Дерево b_2



В лист кладётся среднее остатков

Ансамбль после шага

$$a_2 = a_1 + \eta b_2$$

Ошибка была **1,9**

Стала **0,775**

Квартира 30 м²

$$8 - 0{,}5 \cdot 1 = 7{,}5$$

Порог совпал с порогом первой итерации: половина ошибки была оставлена неисправленной специально, и второму дереву досталась именно она.

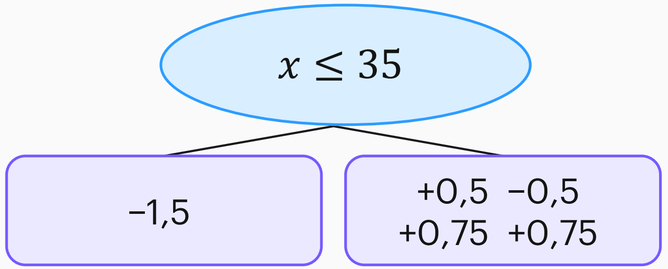
Итерация 3: дерево b_3 на новых остатках

На чём учится дерево

$$r_i = y_i - a_3(x_i)$$



Дерево b_3



среднее −1,5 среднее +0,375

В лист кладётся среднее остатков

Ансамбль после шага

$$a_3 = a_2 + \eta b_3$$

Ошибка была **0,775**

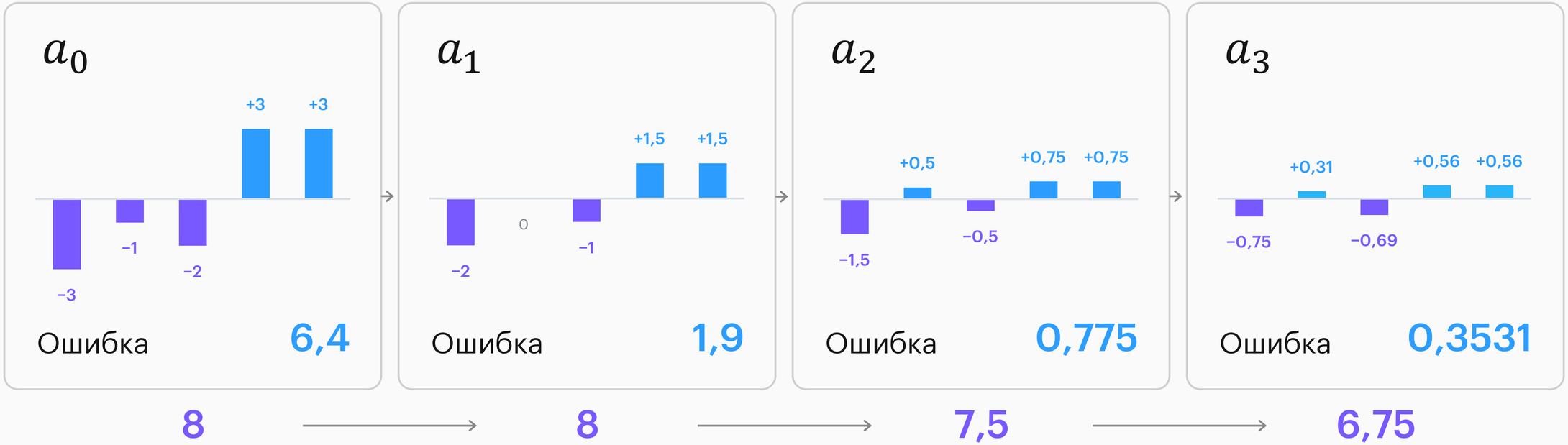
Стала **0,3531**

Квартира 30 м²

$$7{,}5 - 0{,}5 \cdot 1{,}5 = 6{,}75$$

Разбиение сменилось: крупную закономерность первые два дерева уже объяснили, зато на объекте 1 накопилась невязка вдвое крупнее любой другой.

Три шага и падение ошибки



Квартира 30 м²

$$9 + 0{,}5 \cdot (-2 - 1 - 1{,}5) = 6{,}75$$

Ошибка упала примерно в восемнадцать раз, притом что каждое дерево по отдельности почти бесполезно.

Что видно из примера

При квадратичной ошибке дерево учится на остатках

Целевой столбец пересчитывается заново перед каждой итерацией. Все три дерева решали одну задачу регрессии, просто с разными ответами.

Темп не даёт закрыть ошибку за шаг

Поэтому второе дерево повторило разбиение первого, а значения в его листьях оказались вдвое меньше.

Характер работы меняется от шага к шагу

Первые деревья описывают крупную структуру, поздние уточняют детали и переходят к отдельным объектам.

Смещение снимается последовательно

Отдельный пень сильно смещён, и, в отличие от случайного леса, бустинг снижает это смещение.

Алгоритм для квадратичной ошибки

Шаг 0

$a_0 = \bar{y}$ — лучшая константа

Шаги $k = 1, \dots, K$

1. $r_i = y_i - a_{k-1}(x_i)$ — остаток — разница между ответом и текущим предсказанием
2. $b_k = \arg\min_b \sum_{i=1}^{N} (b(x_i) - r_i)^2$ — дерево обучается на остатках вместо целевой переменной
3. $a_k(x) = a_{k-1}(x) + \eta b_k(x)$ — к ансамблю прибавляется доля найденной поправки

Ответ:

$$a_K(x) = a_0 + \eta \sum_{k=1}^{K} b_k(x)$$

Общий алгоритм

01

Пример на регрессии

Три пня на пяти квартирах. Частный случай — регрессия и квадратичная ошибка, каждая величина считается в одну строку.

02

Общий алгоритм

Остатки заменяются антиградиентом, и метод перестаёт зависеть от того, какая функция потерь выбрана.

03

Значения листьев

Разложение потерь до второго порядка позволяет приблизить оптимальное значение листа формулой — G/H.

04

Классификация

Те же формулы в шкале логитов. Меняется не алгоритм, а то, что в него подставляется.

Пример был регрессионный, и ошибка в нём квадратичная. Дальше это ограничение снимается.

От остатков к антиградиенту

$$S_i = -g_i, \qquad g_i = \left.\frac{\partial L(y_i, z)}{\partial z}\right|_{z = a_{k-1}(x_i)}$$

g_i — производная потерь по предсказанию на объекте, s_i — координата антиградиента на нём же

Антиградиент показывает, в какую сторону сдвинуть предсказание, чтобы ошибка убывала быстрее всего.

Приём знаком по неделям 3–4, только там производная бралась по весам, а здесь — по предсказаниям.

Для квадратичной ошибки антиградиент равен остатку.

Значит, счёт по остаткам не был отдельным приёмом: это шаг градиентного спуска, просто не названный своим именем.

Три функции потерь

Функция потерь	$L(y, z)$	**Антиградиент** s_i
Квадратичная	$\frac{1}{2}(y-z)^2$	$y_i - a_{k-1}(x_i)$, то есть остаток
Абсолютная	$\lvert y-z \rvert$	$\operatorname{sign}(y_i - a_{k-1}(x_i))$
Логистическая	$\ln(1+e^z) - yz$	$y_i - p_i$

Направление такой вектор задаёт всегда, а его длина не обязательно равна оптимальному шагу.
Нагляднее всего на абсолютной ошибке: все координаты равны ±1 независимо от того, насколько велик остаток.

Абсолютная ошибка: в точке нулевой ошибки производной нет, там можно выбрать субградиент 0.

Почему нельзя просто прибавить антиградиент

Шаг по антиградиенту

$$a_k(x_i) = a_{k-1}(x_i) + \eta s_i$$

Определён только в N обучающих точках.
Для нового объекта никакого s не считалось:
в производной участвует истинный ответ,
которого там нет.

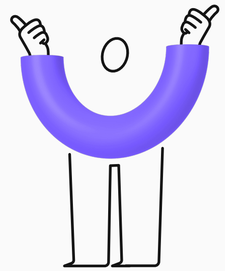
Шаг деревом

$$a_k(x) = a_{k-1}(x) + \eta \hat{b}_k(x)$$

Определён всюду. Дерево кусочно-постоянно
и выдаёт поправку в любой точке
признакового пространства.

$$b_k(x_i) \approx s_i$$

Точное равенство заменяется приближённым — с него и начинается
поиск очередного дерева

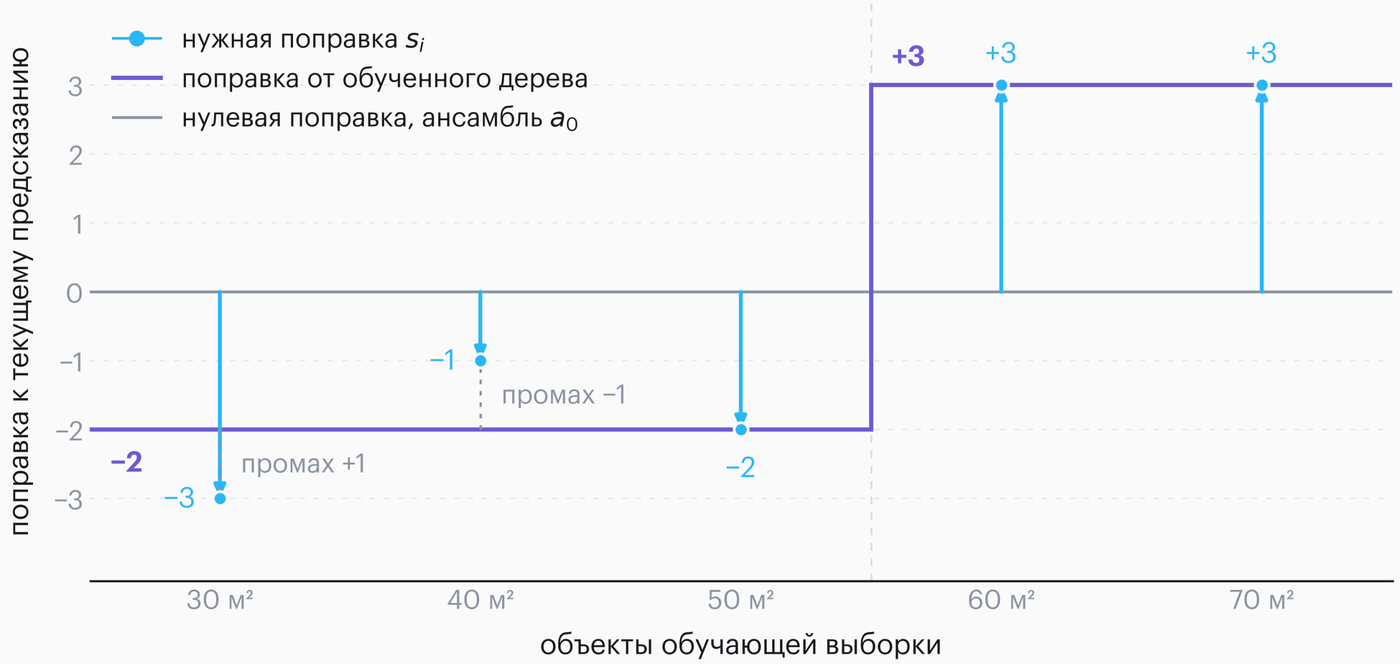


Чем мерить близость к антиградиенту

$$\hat{b}_k = \arg\min_b \sum_{i=1}^{N} (b(x_i) - s_i)^2$$

Задача сменилась: истинные ответы в ней не участвуют, их место заняли требуемые поправки. Значит, и меру близости можно выбирать отдельно.

Это в точности та задача, которую решает обычное регрессионное дерево. Дерево подгоняется по MSE всегда, какой бы ни была L.



Антиградиент задаёт по одному числу на объект. Дерево умеет отвечать только ступенькой и подгоняется к этим числам по квадратичной ошибке.

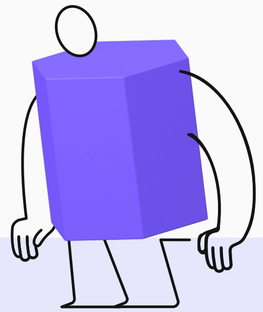
Откуда в названии слово «градиентный»

→ $-\nabla L = (s_1, \dots, s_N)$

направление наискорейшего убывания потерь — по одному числу на каждый обучающий объект

→ $\hat{b}_k = \arg\min_b \sum_{i=1}^{N} (b(x_i) - s_i)^2$

дерево, чьи ответы ближе всего к этому вектору



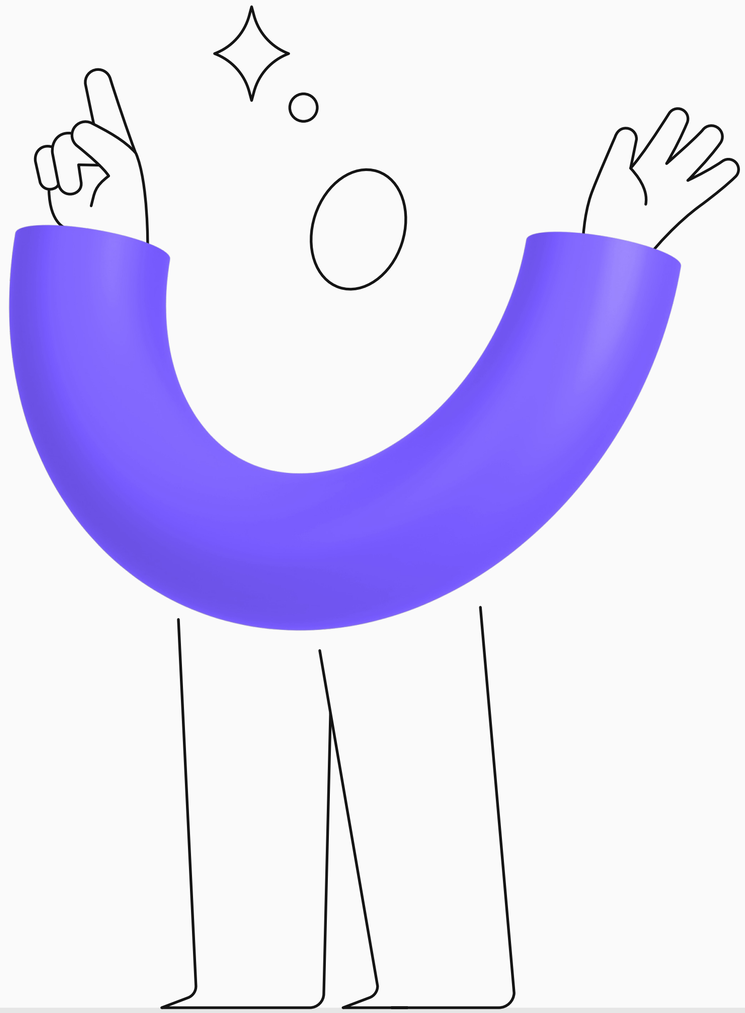
$-\nabla L \to$ проекция на класс деревьев $\to b_k$

поправку задаёт градиент функции потерь

Ограничение идёт от постановки задачи: MSE выбрана мерой близости к антиградиенту.
О возможностях дерева эта мера ничего не говорит.

Для этой записи нужна производная loss по предсказанию. Для негладких функций, например MAE, используют субградиенты.

Собираем алгоритм градиентного бустинга в одном месте



Градиентный бустинг: постановка задачи

Дано: обучающая выборка и функция потерь $L(y, z)$, дифференцируемая по второму аргументу.

$$a_k(x) = a_{k-1}(x) + \eta b_k(x)$$

b_k — базовая модель произвольного вида, η — темп обучения

Задача на шаге k: найти базовую модель, которая сильнее всего уменьшит суммарные потери ансамбля.

$$b_k = \arg\min_b \sum_{i=1}^{N} L\big(y_i, a_{k-1}(x_i) + b(x_i)\big)$$

- Напрямую задача не решается: минимум ищется по функции, и точной формулы у него нет.
- Приближённой заменой служит шаг 2 алгоритма: точный минимум по L заменяется приближением антиградиента.
- Схема к дереву не привязана. На месте b может стоять дерево, линейная или любая другая модель, способная приблизить заданный вектор чисел.

Алгоритм градиентного бустинга

Шаг 0

$$a_0 = \arg\min_c \sum_{i=1}^{N} L(y_i, c)$$

лучшая константа

Шаги $k = 1, \ldots, K$.

1. $$s_i = -\left.\frac{\partial L(y_i, z)}{\partial z}\right|_{z=a_{k-1}(x_i)}$$

   антиградиент — куда сдвинуть предсказание на каждом объекте

2. $$b_k = \arg\min_b \sum_{i=1}^{N} (b(x_i) - s_i)^2$$

   дерево обучается приближать антиградиент по квадратичной ошибке

3. $$a_k(x) = a_{k-1}(x) + \eta b_k(x)$$

   к ансамблю прибавляется доля найденной поправки

Ответ: $a_K(x)$

Регрессия

$= \bar{y}$

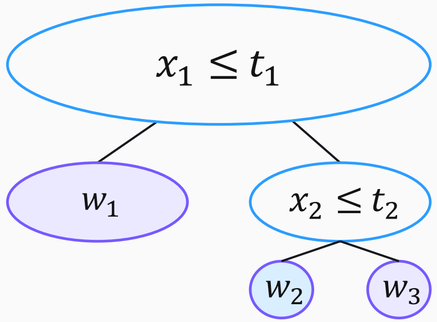
$= y_i - a_{k-1}(x_i)$

Дерево изнутри: структура и значения

$$b_k(x) = \sum_{j=1}^{T} w_j \, \mathbb{I}\big(x \in R_j\big)$$

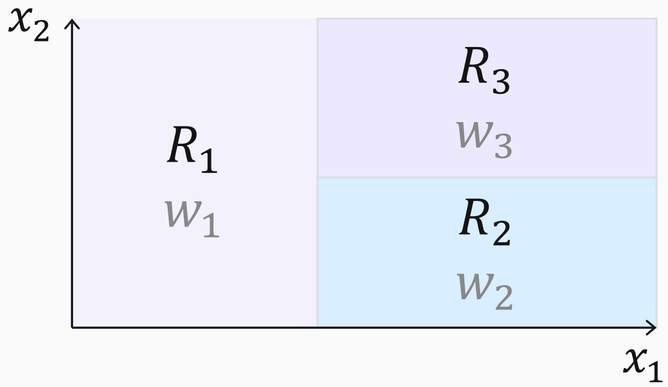
R_j — j-й лист, то есть область признакового пространства,
w_j — число, лежащее в этом листе.

Структура:
где проходят
границы областей



Структура — правила ветвления, значение в листе — число-поправка для всех попавших туда объектов.

Значения:
какое
число стоит
в области



Листья не пересекаются: у объекта ровно один индикатор равен единице.
Темп обучения в эту запись не входит.

Что на самом деле делает сплит

Градиентный бустинг

Где поставить сплит

Структура: листья $R_1, R_2, \ldots$

Что положить в лист

Значение w_j

MSE на антиградиентах

Настоящая функция потерь L

Собрать в лист объекты с похожими поправками

Выбрать величину общей поправки

Сумма по объектам распадается на сумму по листьям

$$b_k(x) = \sum_{j=1}^{T} w_j \, \mathbb{I}\big(x \in R_j\big)$$

Структура задаёт листья R_j, значения — числа w_j

$$\sum_{i=1}^{N} L\left(y_i, a_{k-1}(x_i) + b_k(x_i)\right) = \sum_{j=1}^{T} \sum_{i \in R_j} L\left(y_i, a_{k-1}(x_i) + w_j\right)$$

Слева сумма по объектам, справа — сначала по листьям, а внутри листа — по попавшим в него объектам

$$w_j^* = \arg\min_w \sum_{i \in R_j} L\left(y_i, a_{k-1}(x_i) + w\right)$$

Каждый лист минимизируется сам по себе, по одной переменной

Листья не пересекаются — объект попадает ровно в одну внутреннюю сумму.

Структура уже зафиксирована — ищутся только числа, и задача по T переменным решается по одной за раз.

Какое число положить в лист

Лист R_j

 квартира 1, 30 м² **нужно −3** $+w_j$

 квартира 1, 30 м² **нужно −1** $+w_j$

 квартира 1, 30 м² **нужно −2** $+w_j$

Всем трём прибавляется одно и то же число.

Что выбрать в качестве w_j

$$w_j^* = \arg\min_w \sum_{i \in R_j} L\left(y_i, a_{k-1}(x_i) + w\right)$$

Здесь стоит настоящая функция потерь, а не квадратичная: структура уже выбрана, остаётся подобрать число для готовой группы объектов.

Задача по T переменным распадается на T независимых одномерных — веса листьев не связаны между собой ничем.

Алгоритм бустинга над деревьями

Шаг 0

$$a_0 = \arg\min_c \sum_{i=1}^{N} L(y_i, c)$$

Шаги $k = 1, \dots, K$

1. $s_i = -\left.\dfrac{\partial L(y_i, z)}{\partial z}\right|_{z=a_{k-1}(x_i)}$
2. $b_k = \arg\min_b \sum_{i=1}^{N} (b(x_i) - s_i)^2$
3. $a_k(x) = a_{k-1}(x) + \eta \sum_{j=1}^{T} w_j \mathbb{I}(x \in R_j)$

Ответ:

$a_K(x)$

Шаг 2а. Структура

$R_1, \dots, R_T$

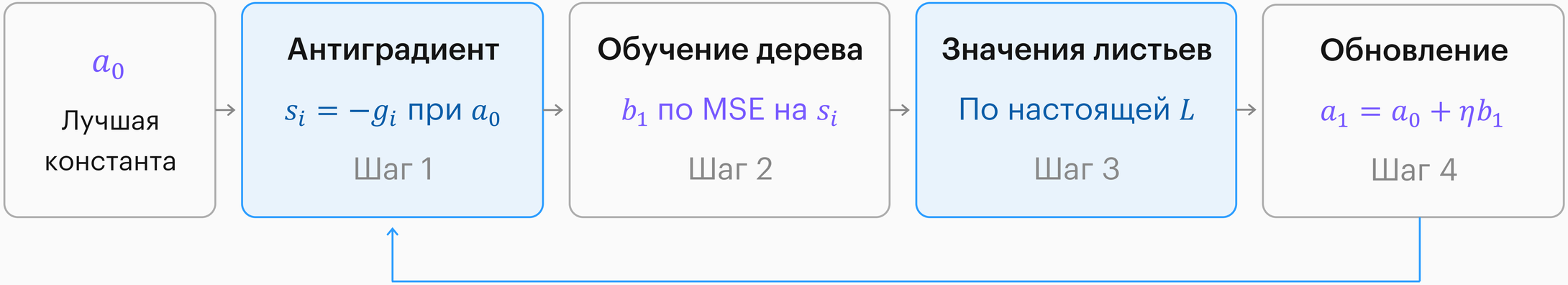
Разбиение ищется по квадратичной ошибке на антиградиенте — как и раньше.

Шаг 2б. Значения листьев

$$w_j = \arg\min_w \sum_{i \in R_j} L(y_i, a_{k-1}(x_i) + w)$$

Число в лист кладётся по настоящей функции потерь.

Каждое дерево учится на антиградиенте



И так K раз: очередное дерево учится
на антиградиенте, взятом в текущей точке ансамбля

Градиентный бустинг итеративно уточняет предсказание: из текущего a_0 вычисляются $s_i = -g_i$, по ним строится дерево b_1, его вклад обновляет предсказание, после чего шаги повторяются.
На каждом шаге дерево обучается на новом таргете s_i, полученном из текущего предсказания.

Значения листьев

01

Пример на регрессии

Три пня на пяти квартирах. Частный случай — регрессия и квадратичная ошибка, каждая величина считается в одну строку.

02

Общий алгоритм

Остатки заменяются антиградиентом, и метод перестаёт зависеть от того, какая функция потерь выбрана.

03

Значения листьев

Разложение потерь до второго порядка позволяет приблизить оптимальное значение листа формулой — G/H.

04

Классификация

Те же формулы в шкале логитов. Меняется не алгоритм, а то, что в него подставляется.

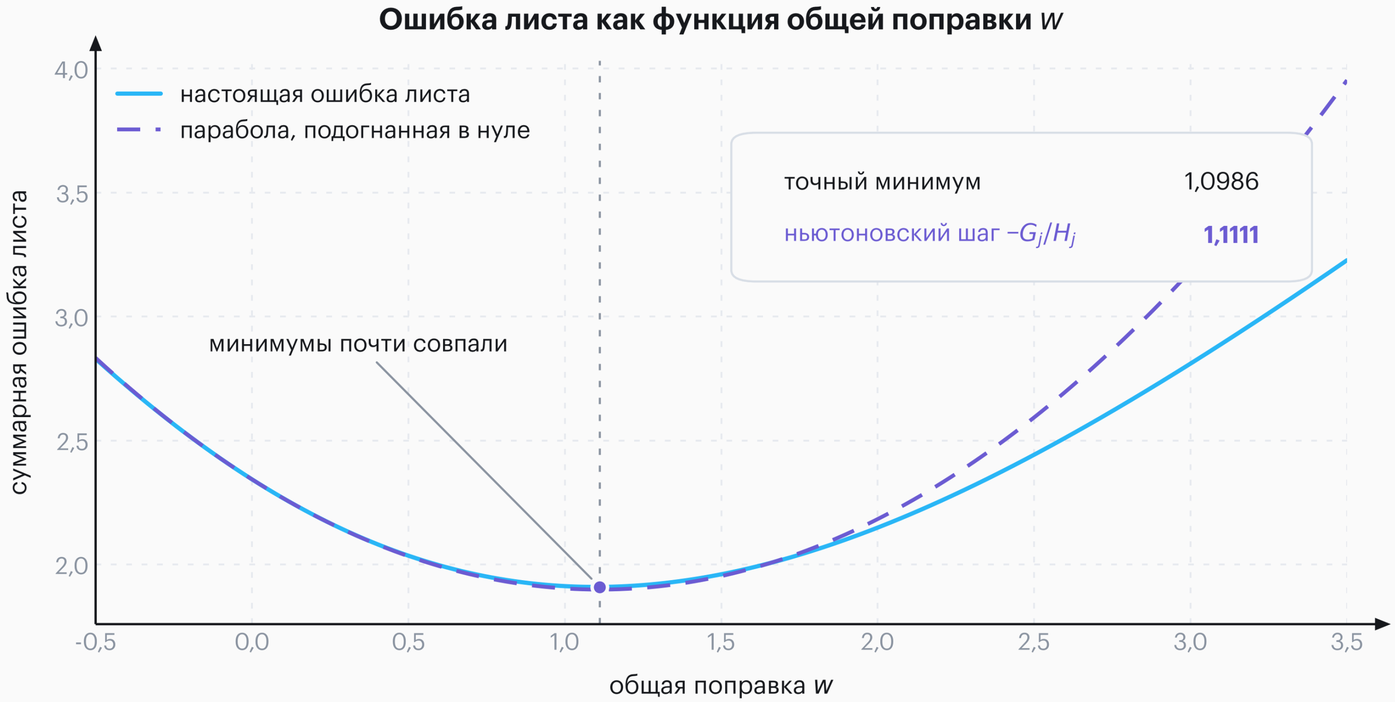
Структуру дерева уже нашли по антиградиентам, а значения листьев задали через argmin настоящей функции потерь. Теперь приближаем этот argmin одним шагом Ньютона.

Заменим сложную функцию параболой

Для каждого листа нужно выбрать **одно значение поправки**, которое лучше всего уменьшает ошибку на объектах этого листа.

Градиент учитывает **наклон** функции потерь,
но не её **кривизну**.

При одинаковом наклоне в точке величина подходящей поправки зависит от того, насколько быстро меняется этот наклон, — то есть от кривизны функции.



У параболы совпадают с исходной функцией значение, наклон и кривизна в текущей точке.

Разложение потерь в листе до второго порядка

→ $L(y_i, a_{k-1}(x_i) + w) \approx L\big(y_i, a_{k-1}(x_i)\big) + g_i w + \frac{1}{2} h_i w^2$

Разложение Тейлора по поправке w около текущего предсказания

→ $g_i = \frac{\partial L(y_i, z)}{\partial z}, \qquad h_i = \frac{\partial^2 L(y_i, z)}{\partial z^2}$

Обе производные берутся в точке $z = a_{k-1}(x_i)$ — там, где ансамбль стоит сейчас

→ $\sum_{i \in R_j} L\left(y_i, a_{k-1}(x_i) + w\right) \approx \text{const} + \underbrace{\sum_{i \in R_j} g_i}_{G_j} w + \frac{1}{2} \underbrace{\sum_{i \in R_j} h_i}_{H_j} w^2$

Складываем по объектам одного листа — поправка w у них общая, а суммы наклонов и кривизн получают имена G_j и H_j

Первое слагаемое не зависит от w: это потери уже построенного ансамбля. Для каждого листа остаётся минимизировать выражение через G_j и H_j, чтобы найти его значение.

Минимум параболы даёт вес листа

$$\frac{d}{dw}\left[G_j w + \frac{1}{2} H_j w^2\right] = G_j + H_j w = 0$$

Производная по единственной переменной листа приравнивается к нулю.

Формула листа через шаг Ньютона

$$w_j = -\frac{G_j}{H_j}$$

$$G_j w_j + \frac{1}{2} H_j w_j^2 = -\frac{G_j^2}{2H_j}$$

Изменение квадратичного приближения потерь при полном шаге Ньютона

Знак минус разворачивает наклон: где ошибка растёт вместе с предсказанием, поправка идёт вниз. При $H_j > 0$ ветви параболы направлены вверх, и найденная точка — её минимум.

Величина $G_j^2/(2H_j)$ вернётся в следующей лекции.

Ньютоновский шаг: числитель и знаменатель

$$w_j = -\frac{G_j}{H_j}, \qquad G_j = \sum_{i \in R_j} g_i\,, \qquad H_j = \sum_{i \in R_j} h_i$$

G_j — суммарный наклон ошибки по объектам листа, H_j — суммарная кривизна по тем же объектам

Числитель говорит, в какую сторону двигать предсказания и насколько сильно объекты этого требуют.

Знаменатель — насколько чувствительна ошибка к движению. Большой H_j означает, что дно рядом и его легко проскочить.

Такой шаг называют **ньютоновским**.
В следующей лекции G_j и H_j будут использоваться в XGBoost.

Требование к L выросло: нужны две производные.
Для квадратичной ошибки и логлосса это выполняется, для модуля отклонения — уже нет.

Классификация

01

Пример на регрессии

Три пня на пяти квартирах. Частный случай — регрессия и квадратичная ошибка, каждая величина считается в одну строку.

02

Общий алгоритм

Остатки заменяются антиградиентом, и метод перестаёт зависеть от того, какая функция потерь выбрана.

03

Значения листьев

Разложение потерь до второго порядка позволяет приблизить оптимальное значение листа формулой — G/H.

04

Классификация

Те же формулы в шкале логитов. Меняется не алгоритм, а то, что в него подставляется.

Все формулы выведены для регрессии. Дальше те же формулы работают в шкале логитов.

Классификация: что меняется построчно

	Регрессия, квадратичная	Классификация, логлосс
Что предсказывает ансамбль	сам ответ y	логит z, вероятность из сигмоиды
Функция потерь	$\frac{1}{2}(y-z)^2$	$\ln(1+e^z)-yz$
Градиент g_i	$a_{k-1}(x_i)-y_i$	p_i-y_i
Кривизна h_i	1	$p_i(1-p_i)$
Начальное приближение	среднее ответов	$\ln\frac{\bar{y}}{1-\bar{y}}$

Алгоритм остаётся прежним, меняются подставляемые в него формулы.

Ансамбль складывает поправки к логиту

→ $$z(x) = a_0 + \eta \sum_{k=1}^{K} b_k\,(x)$$

Деревья складываются в шкале логитов — там сумма ничем не ограничена

→ $$p(x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Вероятность появляется только на выходе, сигмоидой от логита

→ $$a_0 = \ln \frac{\bar{y}}{1 - \bar{y}}$$

Логит доли положительных объектов — лучшая константа для логлосса

Сумма деревьев — произвольное вещественное число, и вероятностью она быть не может.

Логит пробегает всю прямую, складывать деревья ему ничто не мешает.

Производные логлосса по логиту

$$L(y, z) = \ln(1 + e^z) - yz$$

z — логит, который выдаёт ансамбль,
y — метка 0 или 1

$$g_i = \frac{\partial L}{\partial z} = \frac{e^z}{1 + e^z} - y_i = p_i - y_i$$

Наклон — разность предсказанной вероятности и метки

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Вероятность положительного класса

$$h_i = \frac{\partial^2 L}{\partial z^2} = p_i(1 - p_i)$$

Кривизна — производная сигмоиды по логиту

Дифференцирование шло по логиту, а ответ выразился через вероятность. Антиградиент $y_i - p_i$ выглядит как обычный остаток, но остатком в шкале вероятностей не является.

Формула листа в классификации

$$w_j^* = -\frac{G_j}{H_j} = \frac{\sum_{i \in R_j}(y_i - p_i)}{\sum_{i \in R_j} p_i(1-p_i)}$$

Обе суммы берутся по объектам листа R_j

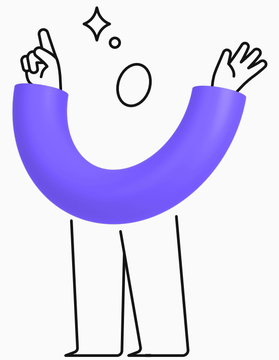
Числитель — сколько вероятности не хватает объектам листа.
Знаменатель — насколько быстро вероятность откликается на сдвиг логита.

Каждый объект входит в обе суммы сразу. При $p = 0{,}5$ он даёт максимум в знаменатель, а уверенная ошибка — заметную невязку в числитель почти без прибавки в знаменатель, и потому сдвигает вес листа сильнее.

Левый лист первого шага — три объекта с метками 0, 1, 1. Доля положительных $\bar{y} = 0{,}4$, поэтому $a_0 = \ln(0{,}4/0{,}6)$, и у всех $p_i = 0{,}4$.

- $G_j = 3 \cdot 0{,}4 - 2 = -0{,}8$
- $H_j = 3 \cdot 0{,}4 \cdot 0{,}6 = 0{,}72$
- $w_j = 0{,}8/0{,}72 = 1{,}1111$

Ньютоновский шаг $1{,}1111$ против точного минимума $\ln 3 = 1{,}0986$, промах $1{,}14\%$.



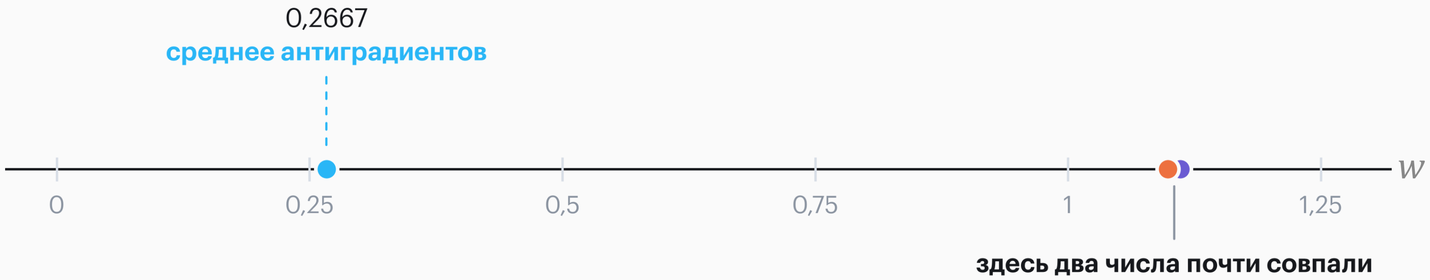
Среднее антиградиентов и шаг Ньютона

Число в тот же лист можно получить двумя способами, и результаты различаются примерно в четыре раза.

Среднее антиградиентов даёт $0{,}2667$, **ньютоновский шаг** $-G_j/H_j$ даёт $1{,}1111$ при точном минимуме $1{,}0986$.

Логлосс падает с $0{,}673$ до $0{,}5043$ с параболическими весами и только до $0{,}6229$ со средними: теряется больше двух третей выигрыша.

Лист с квартирами 1, 2 и 3, метки 0, 1, 1

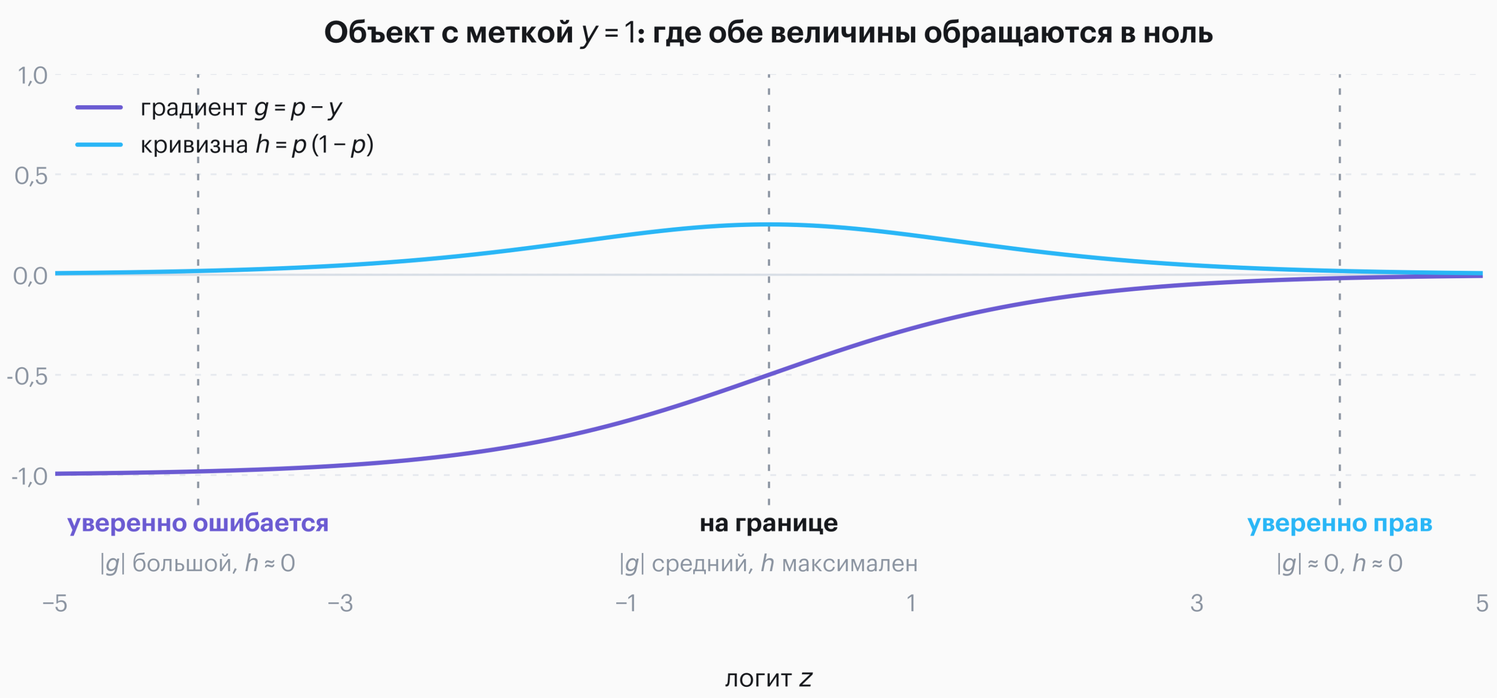


среднее антиградиентов	**0,2667**	без учёта кривизны
ньютоновский шаг $-G_j/H_j$	**1,1111**	с учётом кривизны
точный минимум	**1,0986**	для сверки, промах параболы 1,14 %

Среднее антиградиентов задаёт поправку, но не оптимальный шаг по настоящей функции потерь. Здесь шаг Ньютона ближе к точному минимуму.

Метки пяти объектов: 0, 1, 1, 0, 0. Первый лист — объекты 1–3, второй — 4–5. Темп обучения 0,5. Логлосс посчитан по всем пяти объектам после обновления обоих листьев.

Насыщение сигмоиды



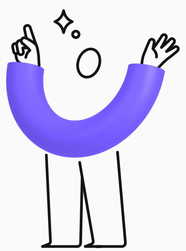
- Кривизна $h_i = p_i(1 - p_i)$ максимальна при $p = 0{,}5$ и падает к нулю по краям.
- На уверенно угаданных объектах модуль градиента близок к нулю.
- При малой суммарной кривизне H_j ньютоновский шаг может оказаться большим.
- Уверенная ошибка, наоборот, тянет вес листа на себя.

Бустинг в sklearn

```python
from sklearn.ensemble import GradientBoostingRegressor

gb = GradientBoostingRegressor(
    loss="squared_error",
    learning_rate=0.5,
    n_estimators=3,
    max_depth=1,
)
gb.fit(X, y)
```

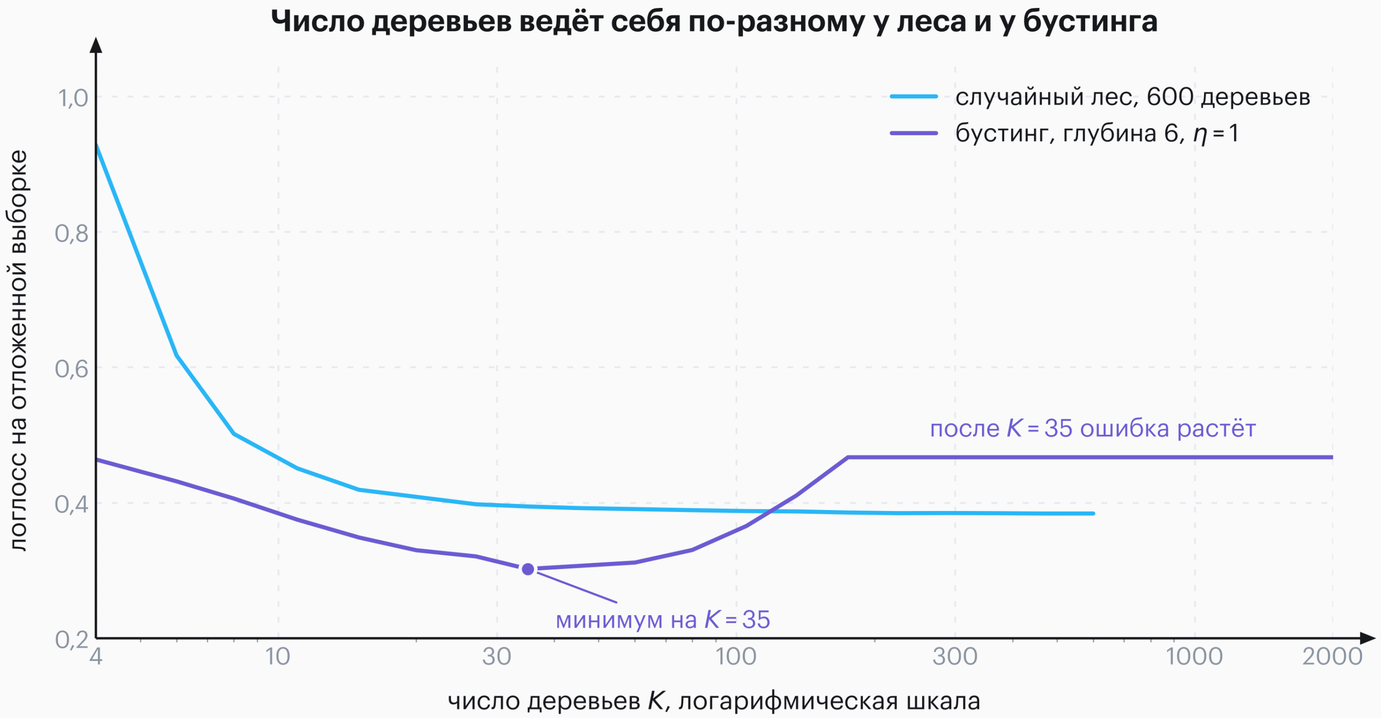
В GradientBoostingRegressor
и GradientBoostingClassifier реализован
тот же алгоритм, что выведен
на предыдущих слайдах.



Параметр	Типичное значение
loss	squared_error — регрессор, log_loss — классификатор
learning_rate	0,1 и меньше
n_estimators	подбирается под темп
max_depth	от 3 до 6
subsample	1,0; меньше даёт стохастический бустинг
n_iter_no_change	None, ранняя остановка выключена

Чем управляют в бустинге

- Темп обучения (`learning_rate`): чем меньше η, тем больше деревьев нужно.
- Число деревьев (`n_estimators`): подбирают совместно с `learning_rate` по валидации.
- Глубина (`max_depth`): от 3 до 6 задаёт порядок взаимодействий, доступный дереву.
- Подвыборка (`subsample`): доля обучающих объектов, случайно отобранных под очередное дерево.



Лес vs бустинг

	Случайный лес	Градиентный бустинг
Порядок построения	параллельно, деревья независимы	последовательно
Что уменьшает ансамблирование (рабочая интуиция)	разброс	смещение
Какие деревья нужны	глубокие и сильные	мелкие, глубины от 3 до 6
Рост числа деревьев	безопасен, выходит на плато	возможно переобучение
Параллелизуемость	полная	только внутри одного дерева
Оценка без отложенной выборки	есть, OOB	валидация или CV; OOB при subsample < 1
Чувствительность к настройке	низкая	высокая
Основные гиперпараметры	n_estimators и max_features	learning_rate, n_estimators и max_depth

OOB-диагностика бустинга отличается от леса: объект мог участвовать в обучении предыдущих деревьев.
В sklearn это oob_improvement_, oob_scores_ и oob_score_.

Лес и бустинг на пяти табличных задачах

Набор данных	Метрика	Лес	Бустинг	Бустинг + подбор
breast_cancer	доля верных	0,986	0,965	0,958
wine	доля верных	0,978	0,978	0,978
digits	доля верных	0,982	0,964	0,976
diabetes	R^2	0,343	0,335	0,390
california_housing	R^2	0,804	0,778	0,825

RandomForest и GradientBoosting из sklearn, классификаторы и регрессоры, параметры по умолчанию. Тест — четверть выборки, `random_state=8`.

Подбор бустинга: `learning_rate` 0,05 и 0,1 × `max_depth` 2, 3 и 4, число деревьев — ранняя остановка по отложенной части обучающей выборки.

- Лес здесь не настраивался и всё равно выиграл или повторил лучший результат на трёх задачах из пяти.
- Подобранный бустинг выигрывает на двух регрессионных: с 0,343 до 0,390 и с 0,804 до 0,825.
- Подбор улучшил результат бустинга на digits и двух регрессионных задачах. На wine результат не изменился, на breast_cancer ухудшился.
- Полный подбор бустинга — 44,5 с, одно обучение леса — 0,7 с: это затраты на две разные процедуры.

Итоги

Лес и бустинг действуют на разные источники ошибки

Усреднение независимых деревьев снижает разброс, поэтому в лесу их делают глубокими. Последовательные поправки снижают смещение, поэтому в бустинге деревья мелкие. Это рабочая интуиция, а не строгое разделение.

Темп не даёт закрыть ошибку за шаг

При квадратичной ошибке правила совпадают, потому что h_i равны единице, а при логистической расходятся в разы.

В классификации бустинг работает в шкале логитов

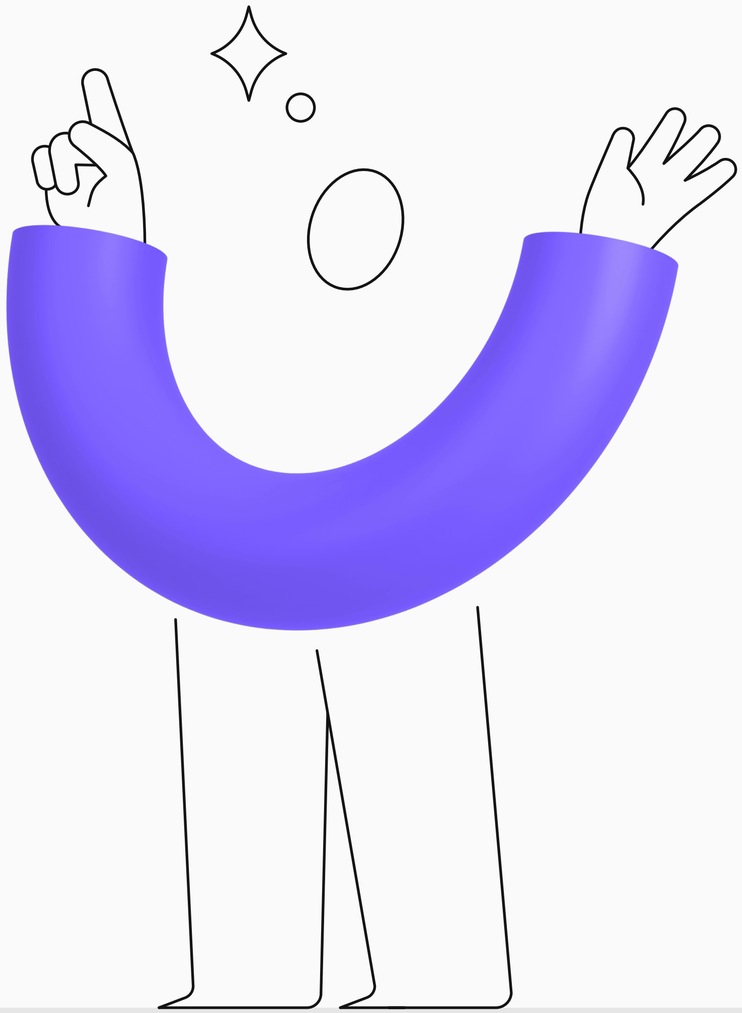
Деревья складываются на логитах, и только на выходе результат превращается в вероятность сигмоидой.

Рост числа деревьев может ухудшить качество

С ростом K ошибка на валидации может начать расти, поэтому число итераций часто выбирают с помощью early stopping.

На следующей лекции — XGBoost, LightGBM и CatBoost

Разберём, как они развивают
и ускоряют базовый градиентный бустинг.





ЦЕНТРАЛЬНЫЙ УНИВЕРСИТЕТ



Вопросы